

Playwright Workshop

Modernes E2E-Testing leicht gemacht



Daniel Sogl - linktr.ee/daniel_sogl

Über den Trainer

Daniel Sogl

Software Architect & Conference Speaker bei
Thinktecture AG

- 🎯 Spezialist für Angular und Generative AI
- 🎤 Tech Speaker & Workshop Trainer
- 🔍 Experte für Test Automation (Playwright, Cypress)
- 📖 Regelmäßiger Konferenz-Speaker



think
tecture

Workshop Details

Organisatorisches

- **Dauer:** 3 Tage, täglich von 9:00 bis 16:00 Uhr
- **Pausen:**
 - Vormittagspause: 10:30 - 10:45 Uhr
 - Mittagspause: 12:00 - 13:00 Uhr
 - Nachmittagspause: 14:30 - 14:45 Uhr
- **Format:** Mix aus Theorie, Live-Coding und praktischen Übungen
- **Interaktion:** Fragen können jederzeit gestellt werden

Ziele des Workshops

- Verständnis von E2E-Testing und dessen Bedeutung
- Beherrschung der Playwright-API und Testmethodik
- Automatisierung komplexer Testszenarien
- Integration von Tests in CI/CD-Pipelines
- Entwicklung stabiler, wartbarer Testsuiten
- Praktische Erfahrung durch hands-on Übungen

Was Sie mitnehmen

-  Fundiertes Wissen über moderne E2E-Testing-Strategien
-  Praktische Erfahrung mit Playwright
-  Best Practices für Test-Automatisierung
-  Tipps und Tricks aus der Praxis
-  Beispiel-Code und Übungsmaterialien
-  Strategien zur CI/CD-Integration

Agenda Tag 1

Vormittag

- **Einführung in Testing**
 - Warum testen wir?
 - Testpyramide & moderne Teststrategien
 - End-to-End Testing im Detail
- **Playwright Grundlagen**
 - Was ist Playwright?
 - Installation & Setup
 - Erste Tests schreiben

Nachmittag

- **Playwright in der Praxis**
 - Locators & Selektoren
 - Interaktionen & Aktionen
 - Assertions & Validierung
- **Debugging & Fehlerbehebung**
 - Playwright Inspector
 - Trace Viewer
 - IDE-Integration
 - Übungen & praktische Anwendung

Grundlagen des Testings

Warum ist Testen wichtig?

Daniel Sogl - linktr.ee/daniel_sogl



Warum Software-Testing?

- **Qualitätssicherung:** Erkennung von Fehlern und Gewährleistung der Funktionalität
- **Risikominimierung:** Frühzeitige Identifizierung kritischer Probleme
- **Kosteneffizienz:** Günstigere Fehlerbeseitigung in frühen Entwicklungsphasen
- **Kundenzufriedenheit:** Sicherstellung einer positiven Nutzererfahrung
- **Vertrauen:** Aufbau von Vertrauen in die Anwendung
- **Kontinuierliche Verbesserung:** Basis für iterative Weiterentwicklung

Testarten im Überblick

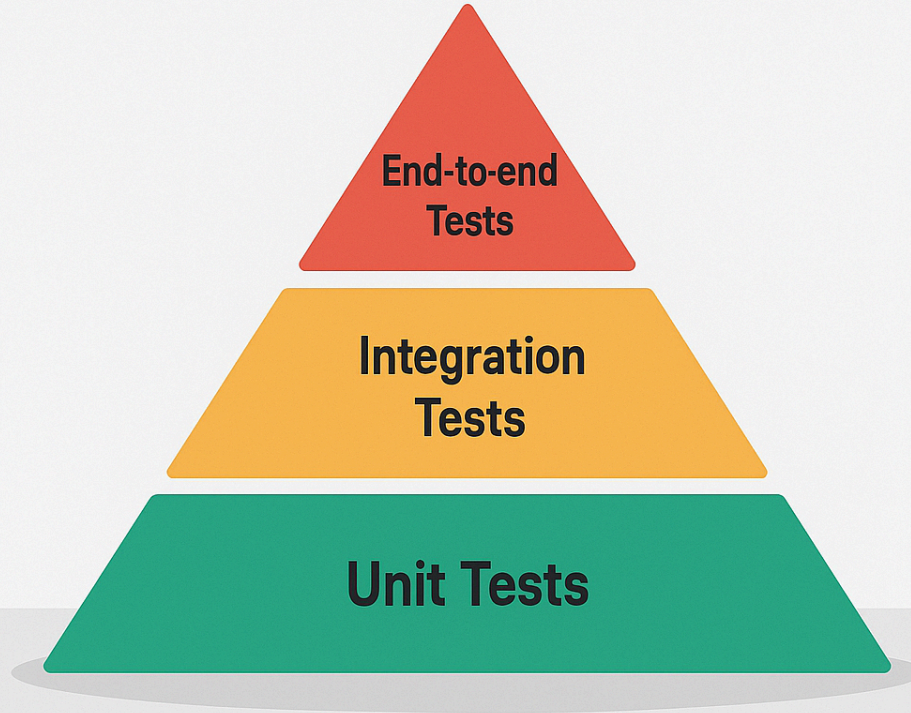
Nach Ebene

- **Unit-Tests:** Testen einzelner Komponenten/Funktionen
- **Integrations-Tests:** Testen des Zusammenspiels von Komponenten
- **System-Tests:** Testen des Gesamtsystems
- **End-to-End-Tests:** Testen ganzer Anwendungsabläufe

Nach Fokus

- **Funktional:** Überprüfung der Funktionalität
- **Nicht-funktional:** Performance, Sicherheit, Usability
- **Regression:** Sicherstellung, dass neue Änderungen keine bestehenden Funktionen beeinträchtigen
- **Akzeptanz:** Überprüfung der Anforderungserfüllung

TESTING PYRAMID



Grenzen klassischer Teststrategien

- Nicht alle Systeme passen zur Pyramidenlogik
- Mobile und Microservice-Architekturen benötigen oft andere Gewichtung
- Fokus zu stark auf Isolierung kann Integrationsfehler übersehen
- UI-Tests oft zu wenig berücksichtigt

End-to-End Testing

- Simuliert reale Benutzerinteraktionen
- Testet die gesamte Anwendung inkl. aller Systeme und Abhängigkeiten
- Validiert den gesamten Funktionsfluss der Anwendung
- **Genau hier setzt Playwright an!**

Vorteil

E2E-Tests validieren die Anwendung aus Nutzerperspektive, genau wie echte Benutzer damit interagieren würden.

Herausforderung

E2E-Tests können langsamer und anfälliger für Instabilitäten sein.

Herausforderungen beim manuellen Testing

- **Zeitaufwand:** Wiederholtes Testen ist zeitintensiv
- **Konsistenz:** Schwierigkeiten, Tests immer identisch durchzuführen
- **Skalierbarkeit:** Begrenzte Kapazität für umfangreiche Testszenarien
- **Dokumentation:** Aufwändige Protokollierung von Testergebnissen
- **Regression:** Hoher Aufwand bei wiederholten Tests nach Änderungen
- **Menschlicher Faktor:** Anfälligkeit für Fehler und Übersehen von Details

Vorteile automatisierter Tests

- **Effizienz:** Schnellere Ausführung als manuelle Tests
- **Wiederholbarkeit:** Identische Ausführung in jeder Testumgebung
- **Konsistenz:** Gleiche Testbedingungen bei jedem Durchlauf
- **Abdeckung:** Möglichkeit, mehr Testszenarien abzudecken
- **Frühe Fehlererkennung:** Schnelles Feedback in der Entwicklung
- **CI/CD-Integration:** Automatische Ausführung als Teil der Pipeline
- **Dokumentation:** Tests dienen als lebende Dokumentation

Unit-Tests – Grundlagen

Ziel: Testen kleinster, isolierter Code-Einheiten (z.B. Funktionen, Methoden, Klassen).

Fokus: Überprüfung der Logik einer einzelnen Einheit ohne externe Abhängigkeiten (oft durch Mocks/Stubs ersetzt).

- Isolierte Tests einzelner Funktionalitäten
- Externe Abhängigkeiten werden durch Mocks/Stubs ersetzt
- Basis der Testpyramide mit der größten Testanzahl
- Schnelle Ausführung und einfaches Setup

Unit-Tests – Vor- und Nachteile

Vorteile

- **Schnell:** Sehr schnelle Ausführung
- **Präzise Fehlerlokalisierung:** Fehler können leicht einer spezifischen Einheit zugeordnet werden
- **Fördert gutes Design:** Zwingt Entwickler zu modularem, entkoppeltem Code
- **Frühes Feedback:** Schnelle Rückmeldung während der Entwicklung

Nachteile

- **Keine Integrationsgarantie:** Testen nicht das Zusammenspiel verschiedener Einheiten
- **"Mock Hell":** Kann zu übermäßigem Mocking führen, das von der Realität abweicht
- **Blind für Systemfehler:** Finden keine Fehler, die nur im Gesamtsystem auftreten

Integrations-Tests – Grundlagen

Ziel: Testen des Zusammenspiels mehrerer Komponenten oder Module.

Fokus: Überprüfung der Schnittstellen und Datenflüsse zwischen integrierten Teilen des Systems.

Beispiele: Interaktion zwischen Service-Schicht und Datenbank, Kommunikation zwischen Microservices.

- **Realitätsnäher als Unit-Tests:** Testen das tatsächliche Zusammenspiel von Komponenten
- **Finden Schnittstellenprobleme:** Decken Fehler in der Kommunikation zwischen Modulen auf
- **Höheres Vertrauen:** Geben mehr Sicherheit über die Korrektheit der Integration

Integrations-Tests – Vor- und Nachteile

Vorteile

- **Realitätsnäher als Unit-Tests:** Testen das tatsächliche Zusammenspiel von Komponenten
- **Finden Schnittstellenprobleme:** Decken Fehler in der Kommunikation zwischen Modulen auf
- **Höheres Vertrauen:** Geben mehr Sicherheit über die Korrektheit der Integration

Nachteile

- **Langsamer als Unit-Tests:** Benötigen oft mehr Setup und Laufzeit
- **Schwierigere Fehleranalyse:** Fehlerursachen können komplexer zu finden sein als bei Unit-Tests
- **Abhängigkeiten:** Erfordern oft laufende externe Systeme (Datenbanken, andere Services)

E2E Tests – Überblick

Ziel: Simulation vollständiger Benutzerabläufe durch die gesamte Anwendung.

Fokus: Validierung des gesamten Systemflusses aus der Perspektive des Endbenutzers, inklusive UI, Backend, Datenbanken und externer Integrationen.

Beispiele: Einloggen, Artikel zum Warenkorb hinzufügen, Kasse, Ausloggen.

- Testen der Anwendung aus Benutzerperspektive
- Validierung vollständiger Geschäftsprozesse
- Überprüfung aller Systemkomponenten im Zusammenspiel
- Simulation realer Nutzungsszenarien

E2E Tests – Vor- und Nachteile

Vorteile

- **Höchstes Vertrauen:** Testen die Anwendung so, wie ein Benutzer sie erleben würde
- **Validieren komplette Features:** Stellen sicher, dass ganze Geschäftsabläufe funktionieren
- **Decken systemweite Probleme auf:** Finden Fehler, die nur im Zusammenspiel aller Teile auftreten
- **Playwright hilft, diese Vorteile zu maximieren!**

Nachteile

- **Langsam:** Deutlich längere Ausführungszeiten als Unit- oder Integrationstests
- **Instabil ("Flaky"):** Anfällig für Probleme durch Timing, Netzwerk, UI-Änderungen oder Testdaten
- **Aufwändig:** Komplexer in der Erstellung und Wartung
- **Schwierige Diagnose:** Fehlerursachen können schwer zu identifizieren sein
- **Playwright hilft, diese Nachteile zu minimieren!**

Teststrategie: Nicht nur Theorie!

- **Warum das Ganze?** Die richtige Strategie spart Zeit & Nerven.
- **Ziel:** Hohe Testabdeckung mit optimalem Aufwand.
- **Playwright im Kontext:** Stärkt massiv die E2E-Säule eurer Tests.
- **Diskussion:** Welche Strategie passt zu eurem Projekt? Wie kann Playwright helfen?

Kritik an der Test-Pyramide – Übersicht

Obwohl ein wertvolles Konzept, stößt die klassische Pyramide an Grenzen:

- **Überbetonung von Unit-Tests?:** Können zu viele isolierte Tests ein falsches Sicherheitsgefühl geben, wenn Integrationspunkte fehlschlagen?
- **Vernachlässigung von Integrationstests?:** Das Zusammenspiel von Komponenten ist oft fehleranfällig und wird möglicherweise unterrepräsentiert.
- **Wird E2E unterschätzt?:** Werden E2E-Tests zu stark reduziert, obwohl sie das Nutzererlebnis am besten abbilden?

Kritik an der Test-Pyramide – Moderne Anforderungen

- **Moderne Architekturen:** Passt das Modell noch perfekt zu Microservices, Frontend-Frameworks oder stark vernetzten Systemen?
- **CI/CD ermöglicht häufige und schnelle Ausführung auch komplexerer Tests**

Hinweis

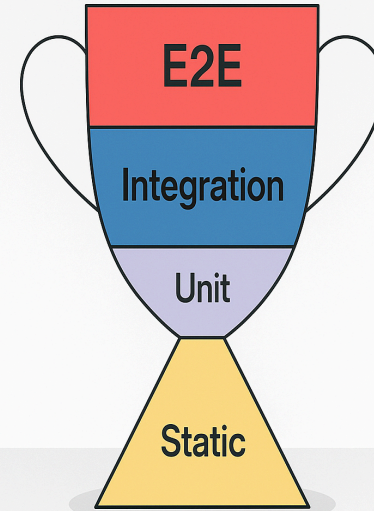
*Die Pyramide ist ein **Modell**, keine starre Regel. Der Kontext (Architektur, Team, Risiken) ist entscheidend.*

Moderne Ansätze: Testing Trophy – Struktur

Die Testing Trophy legt den Fokus auf Integrationstests als wertvollste Testart.

Struktur:

- **Statische Tests** (Basis): Linters, Typ-Checker
- **Unit Tests**: Weniger als in der Pyramide
- **Integration Tests** (Kern): Der größte und wichtigste Teil
- **E2E Tests** (Spitze): Wichtig, aber wenige

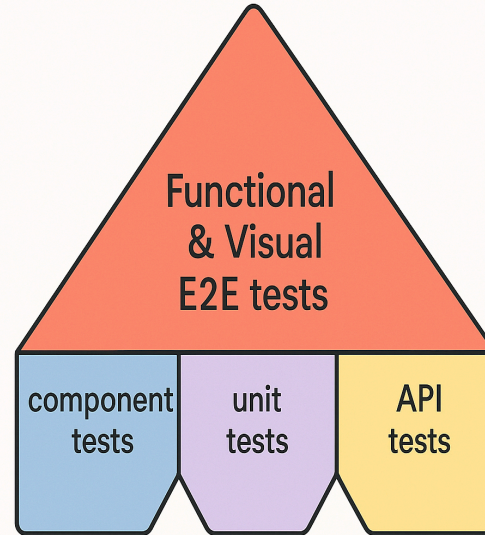


Testing Crab – Struktur & Merkmale

Von Gleb Bahmutov, betont eine **parallele und ausgewogene** Verteilung:

Struktur (Beispiele):

- Component Tests
- API Tests
- Unit Tests
- Functional E2E Tests
- Visual E2E Tests
- Performance Tests
- Security Tests



Testing Crab – Philosophie

Philosophie:

- **Keine Hierarchie:** Alle Testarten sind wichtig und können parallel laufen.
- **Fokus auf Nutzererlebnis:** Starke Betonung von funktionalen und visuellen E2E-Tests.
- **Flexibilität:** Passt sich gut an komplexe UIs und agile Prozesse an.
- **Parallelisierung:** Schnelleres Feedback durch gleichzeitige Ausführung verschiedener Testsuiten.

Vergleich verschiedener Modelle

Modell	Stärke	Ideal für...
Pyramide	Starke Basis, schnell, gute Isolation	Modulare Systeme, Backend-Services
Trophy	Fokus auf Komponenten-Interaktion	Full-Stack-Apps, Systeme mit vielen Abhängigkeiten
Crab	Parallel, ausgewogen, starker E2E/Visual-Fokus	Komplexe UIs, Agilität, schnelles Feedback

Empfehlung: Verstehe die Prinzipien, passe sie an und **sei bereit zur Anpassung!**

Zusammenfassung – Testarten & Strategien

- Keine Teststrategie ist universell richtig
- Kombination aus Unit, Integration und E2E Tests ist meist sinnvoll
- Kontextabhängige Gewichtung ist entscheidend
- Moderne Tools wie Playwright erlauben produktive E2E-Automatisierung
- **Playwright ermöglicht es, E2E-Tests als starken Pfeiler eurer Qualitätssicherung zu etablieren.**

Playwright Grundlagen

Tag 1: Einstieg in Playwright und E2E-Testing



Daniel Sogl - linktr.ee/daniel_sogl

Was ist Playwright? – Überblick & Geschichte

- **Open-Source** End-to-End Testing Framework von **Microsoft** (seit 2020)
- Entwickelt vom Team, das zuvor an Puppeteer arbeitete
- Basierend auf dem Chrome DevTools Protocol
- **Cross-Browser** Testing für Chromium, Firefox, WebKit
- Unterstützt .NET, JavaScript, TypeScript, Python und Java
- Ziel: Einschränkungen bestehender Frameworks überwinden
- Schnelle Entwicklung, wachsende Community

Typische E2E-Testing Schmerzpunkte

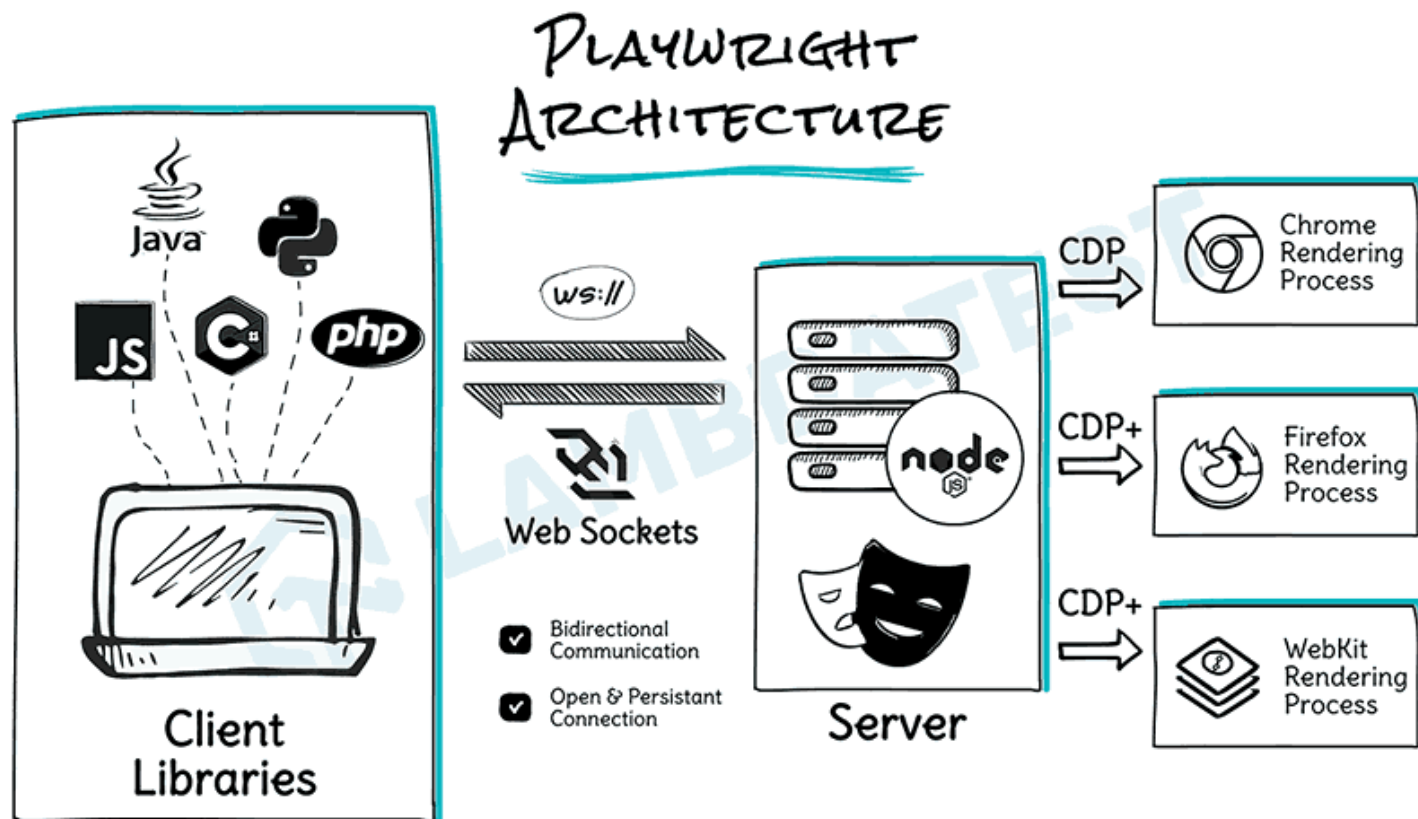
- **Instabile Tests (Flakiness):** Tests schlagen ohne Code-Änderung fehl.
- **Langsames Feedback:** Lange Testausführungszeiten bremsen die Entwicklung.
- **Cross-Browser-Herausforderungen:** Unterschiedliches Verhalten in Browsern.
- **Komplexe Setups:** Schwierige Konfiguration und Wartung der Testumgebung.
- **Schwieriges Debugging:** Fehler in Tests sind schwer zu finden.
- **Mangelnde Zuverlässigkeit:** Tests finden nicht immer alle Fehler.

Kommen Ihnen diese bekannt vor? Playwright wurde entwickelt, um genau diese Probleme zu lösen!

Warum Playwright? Die Lösung für Ihre Test-Herausforderungen

- **Multi-Browser-Support:** Testen Sie konsistent über Chromium, Firefox & WebKit.
- **Auto-Wait & Stabilität:** Intelligente Synchronisation verhindert Flakiness.
- **Geschwindigkeit:** Schnelle Ausführung für zügiges Feedback.
- **Network Interception:** Mocken Sie APIs und kontrollieren Sie das Netzwerkverhalten.
- **Mobile Emulation:** Testen Sie responsive Designs einfach und zuverlässig.
- **Isolation & Zuverlässigkeit:** Browser-Contexts sorgen für unabhängige Tests.
- **Moderne Architekturen:** SPAs, Shadow-DOM, iframes – kein Problem!
- **Traces & Debugging:** Umfangreiche Werkzeuge für schnelle Fehlersuche.

Playwright-Architektur im Überblick



Playwright vs. Cypress

- **Cypress:** Sehr beliebt, einfache Einrichtung, tolle DX, aber nur JS/TS, eingeschränkter Browser-Support, keine Multi-Tab/Window-Tests
- **Playwright:** Multi-Browser, Multi-Language, bessere Parallelisierung, umfangreichere API, schnellere Ausführung, Cross-Domain-Tests

Technische Unterschiede: Cypress vs. Playwright

- **Cypress:** Läuft im Browser, jQuery-ähnliche Syntax, automatische Wartezeiten, eingeschränkte Domain-Wechsel, gute Debug-Tools
- **Playwright:** Browser-übergreifende Architektur, moderne Test-APIs, smartes Auto-Waiting, volle Domain-Kontrolle, Trace-Funktionen, Netzwerk-Kontrolle

Installationsvoraussetzungen

- **.NET SDK** (v9.0 oder höher)
- **PowerShell** (für Browser-Installation)
- Für verschiedene Browser:
 - **Windows:** Keine zusätzlichen Anforderungen
 - **macOS:** Keine zusätzlichen Anforderungen
 - **Linux:** Eventuell fehlende Browser-Abhängigkeiten

Tipp

Playwright installiert automatisch alle erforderlichen Browser.

Installation von Playwright

```
1  # Neues .NET Projekt erstellen
2  dotnet new mstest -n PlaywrightTests
3  cd PlaywrightTests
4
5  # Playwright NuGet Package hinzufügen
6  dotnet add package Microsoft.Playwright.MSTest
7
8  # Projekt kompilieren
9  dotnet build
10
11 # Playwright Browser installieren
12 pwsh bin/Debug/net9.0/playwright.ps1 install
```

Bei der Installation:

- .NET Test Framework wählen (MSTest empfohlen)
- NuGet Package hinzufügen
- Browser installieren
- Projekt kompilieren

Test Generator – Automatische Testgenerierung

- **Was ist der Test Generator?**

- Ein Tool, das automatisch Playwright-Tests generiert, während Sie mit der Anwendung interagieren.
- Starten Sie den Generator mit `pwsh bin/Debug/net9.0/playwright.ps1 codegen <URL> .`

- **Vorteile:**

- Spart Zeit bei der Erstellung von Tests.
- Generiert sofort ausführbaren Code.
- **Hilft bei der Auswahl stabiler, benutzerorientierter Locators.**

- **Demo (noch einmal, weil es so wichtig ist!):**

- Öffnen Sie ein Terminal und führen Sie `pwsh bin/Debug/net9.0/playwright.ps1 codegen IHRE_APP_URL` aus.
- Interagieren Sie mit Ihrer Anwendung, um automatisch Testcode zu generieren.

Demo Projekt auschecken

1. Repository auschecken:

```
git clone https://github.com/danielsogl/playwright-workshop.git
cd playwright-workshop
git checkout dotnet
```

2. Projekt in VS Code öffnen:

```
code .
```

3. Abhängigkeiten installieren:

```
cd PlaywrightTests
dotnet restore
```

4. Demo Tests ausführen:

```
dotnet test
```

Schnellstart: Erster Test mit Codegen

1. Playwright Codegen starten:

```
pwsh bin/Debug/net9.0/playwright.ps1 codegen demo.playwright.dev/todomvc
```

2. Interagieren Sie mit der Seite: Fügen Sie Todos hinzu, bearbeiten Sie sie, löschen Sie sie.

3. Beobachten Sie den generierten Code!

4. Kopieren Sie den Code in eine neue Datei (z.B. Tests/ToDoTests.cs).

5. Test ausführen:

```
dotnet test --filter "ClassName=ToDoTests"
```



Glückwunsch! Ihr erster Playwright Test lief erfolgreich!

Installationsoptionen im Detail

```
1  # MSTest Framework (empfohlen für diesen Workshop)
2  dotnet new mstest -n PlaywrightTests
3
4  # Playwright Package hinzufügen
5  dotnet add package Microsoft.Playwright.MSTest
6
7  # Alternative Test Frameworks
8  # dotnet new nunit -n PlaywrightTests
9  # dotnet add package Microsoft.Playwright.NUnit
10 # dotnet new xunit -n PlaywrightTests
11 # dotnet add package Microsoft.Playwright.Xunit
```

Übung 0 – Projekt-Setup

Ziel: Playwright-Projekt für die Feed App einrichten & prüfen.

Aufgaben:

1. .NET Playwright-Projekt initialisieren (`dotnet new mstest -n PlaywrightTests`).
2. Abhängigkeiten installieren & App starten.
3. Einen ersten Beispieltest schreiben (Startseite öffnen, Titel prüfen).
4. Projektstruktur (`PlaywrightTests.csproj` , `Tests/`) ansehen.

Zeit: 15 Minuten

Projektstruktur

```
1  PlaywrightTests/
2  |— Tests/           # Testdateien
3  |   |— ExampleTests.cs # Testspezifikation
4  |   |— Fixtures/      # Testdaten
5  |— PlaywrightTests.csproj # Projekt-Konfiguration
6  |— .runsettings      # Optionale Browser-Konfiguration
7  |— TestResults/      # Testergebnisse
8  |   |— traces/        # Aufgezeichnete Traces
9  |   |— screenshots/   # Testscreenshots
```

Wichtigste Dateien für den Anfang:

- `PlaywrightTests.csproj` : Projektdatei mit NuGet-Abhängigkeiten.
- `Tests/` : Hier leben Ihre Testskripte.
- `.runsettings` : Optionale Konfiguration für Browser und Test-Einstellungen.

.NET Projekt Setup

```
1  <!-- PlaywrightTests.csproj -->
2  <Project Sdk="Microsoft.NET.Sdk">
3
4      <PropertyGroup>
5          <TargetFramework>net9.0</TargetFramework>
6          <LangVersion>latest</LangVersion>
7          <ImplicitUsings>enable</ImplicitUsings>
8          <Nullable>enable</Nullable>
9      </PropertyGroup>
10
11     <ItemGroup>
12         <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
13         <PackageReference Include="Microsoft.Playwright.MSTest" Version="1.54.0" />
14         <PackageReference Include="MSTest" Version="3.6.4" />
15     </ItemGroup>
16
17     <ItemGroup>
18         <Using Include="Microsoft.VisualStudio.TestTools.UnitTesting" />
19     </ItemGroup>
20
21 </Project>
```


Konfigurationsdatei - Wichtigste Optionen erklärt

- `TargetFramework` : .NET Version für Ihre Tests (z.B. `net6.0`).
- `PackageReference` : NuGet Packages für Playwright und Test Framework.
- `ImplicitUsings` : Automatische Using-Statements für häufige Namespaces.
- **Konfiguration:** Erfolgt über `runsettings.xml` oder Umgebungsvariablen.

Weitere Optionen (`timeout` , `reporter` , `trace` etc.) werden wir später genauer betrachten.

Optionale Konfiguration - .runsettings

```
1  <!-- .runsettings -->
2  <?xml version="1.0" encoding="utf-8"?>
3  <RunSettings>
4    <Playwright>
5      <!-- Browser Configuration -->
6      <BrowserName>firefox</BrowserName>
7      <!-- Supported browsers: chromium, firefox, webkit -->
8
9      <!-- Locale Configuration -->
10     <Locale>en-US</Locale>
11     <!-- Examples: en-US, de-DE, fr-FR, es-ES, ja-JP, zh-CN -->
12
13     <!-- Launch Options -->
14     <LaunchOptions>
15       <ViewportSize>
16         <Width>1280</Width>
17         <Height>720</Height>
18       </ViewportSize>
19     </LaunchOptions>
20   </Playwright>
21 </RunSettings>
```

.NET Playwright - Wichtige Unterschiede

- **Keine playwright.config.ts:** .NET verwendet `.runsettings` oder Umgebungsvariablen
- **Test-Framework Integration:** Direkt in MSTest integriert
- **Base Classes:** `PageTest` , `ContextTest` , `BrowserTest` für verschiedene Scopes
- **Konfiguration im Code:** Über `ContextOptions()` Methode möglich

```
1  public override BrowserNewContextOptions ContextOptions()
2  {
3      return new BrowserNewContextOptions
4      {
5          ViewportSize = new() { Width = 1920, Height = 1080 },
6          BaseURL = "http://localhost:3000"
7      };
8  }
```

Browser-Konfiguration in .NET

```
1  # Verschiedene Browser per CLI
2  dotnet test -- Playwright.BrowserName=chromium
3  dotnet test -- Playwright.BrowserName=firefox
4  dotnet test -- Playwright.BrowserName=webkit
5
6  # Oder mit .runsettings Dateien
7  dotnet test --settings chromium.runsettings
8  dotnet test --settings firefox.runsettings
```

```
1  <!-- chromium.runsettings -->
2  <RunSettings>
3    <Playwright>
4      <BrowserName>chromium</BrowserName>
5    </Playwright>
6  </RunSettings>
```

Wichtige Konfigurationsoptionen

- **TargetFramework:** .NET Version (net9.0 empfohlen)
- **MaxCpuCount:** Anzahl paralleler Test-Threads
- **Playwright Settings:** Browser-Konfiguration
 - **BrowserName:** Browser-Auswahl (chromium, firefox, webkit)
 - **LaunchOptions:** Browser-Startoptionen (Headless, Channel)
- **Base Test Classes:** `PageTest` , `ContextTest` , `BrowserTest`
- **Konfiguration:** Über `.runsettings` , CLI oder Code möglich

Erster Test – Codeübersicht

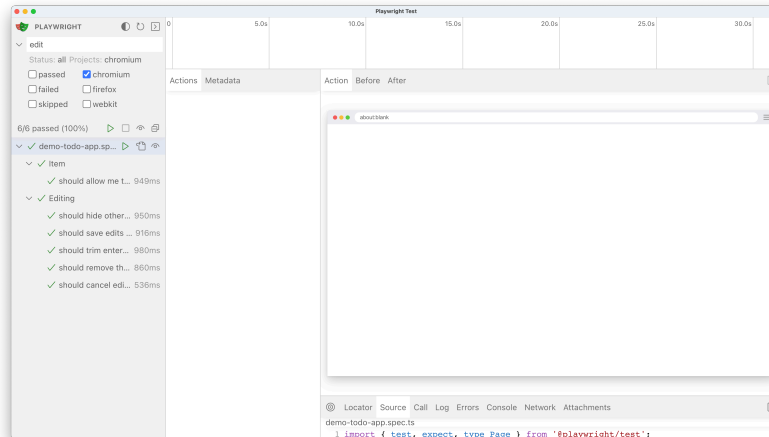
```
1  using Microsoft.Playwright.MSTest;
2  using System.Text.RegularExpressions;
3
4  [TestClass]
5  public class StartseiteTest : PageTest
6  {
7      [TestMethod]
8      public async Task StartseiteHatRichtigenTitel()
9      {
10         // Seite öffnen
11         await Page.GotoAsync("https://example.com");
12
13         // Titel prüfen
14         await Expect(Page).ToHaveTitleAsync(new Regex("Example Domain"));
15
16         // Text auf der Seite prüfen
17         var heading = Page.Locator("h1");
18         await Expect(heading).ToHaveTextAsync("Example Domain");
19     }
20 }
```

Teststruktur im Detail

```
1  using Microsoft.Playwright.MSTest;
2  using System.Text.RegularExpressions;
3
4  // Testgruppe definieren
5  [TestClass]
6  public class StartseitenTests : PageTest
7  {
8      // Setup vor jedem Test
9      [TestInitialize]
10     public async Task Setup()
11     {
12         await Page.GotoAsync("https://example.com");
13     }
14
15     // Einzelner Test
16     [TestMethod]
17     public async Task HatRichtigenTitel()
18     {
19         await Expect(Page).ToHaveTitleAsync(new Regex("Example Domain"));
20     }
21 }
```

Erster Test – Ergebnisse

```
1  # Test ausführen
2  dotnet test
3
4  # Test mit spezifischer Konfiguration ausführen
5  dotnet test --settings chromium.runsettings
6
7  # Test in spezifischem Browser ausführen
8  dotnet test -- Playwright.BrowserName=chromium
```



Testausführung – Erweiterte Optionen

```
1  # Bestimmte Testklasse ausführen
2  dotnet test --filter "ClassName=LoginTests"
3
4  # Tests mit bestimmtem Namen ausführen
5  dotnet test --filter "Name~Login"
6
7  # Tests im Watch-Modus ausführen (bei Änderungen neu starten)
8  dotnet watch test
9
10 # Headed-Modus (sichtbare Browser)
11 dotnet test -- Playwright.LaunchOptions.Headless=false
12
13 # Debugging mit Inspector
14 PWDEBUG=1 dotnet test
15
16 # Parallele Ausführung konfigurieren
17 dotnet test --parallel
```

Headless vs. Headed Testing

Headless Testing (Standard)

- Schnellere Ausführung
- Ressourcenschonender
- Ideal für CI/CD
- Keine UI-Interaktion nötig
- Geeignet für Regressionstests

Headed Testing (Sichtbar)

- Bessere Visualisierung
- Einfacheres Debugging
- Langsamer als Headless
- Mehr Ressourcenverbrauch
- Geeignet für Entwicklung und Fehlersuche

```
1  # Headed-Modus aktivieren
2  dotnet test -- Playwright.LaunchOptions.Headless=false
3
4  # In der Konfiguration (.runsettings)
5  <Playwright>
6    <LaunchOptions>
7      <Headless>false</Headless>
8    </LaunchOptions>
9  </Playwright>
```

Locators

Was sind Locators?

- Locators sind Ausdrücke, mit denen Playwright Elemente auf einer Webseite findet und anspricht.
- Sie ermöglichen es, gezielt mit Buttons, Feldern, Texten usw. zu interagieren.
- Playwright unterstützt verschiedene Locator-Typen: Text, Rolle, Label, Test-ID, CSS, XPath u.v.m.
- Gute Locators machen Tests stabil, lesbar und wartbar.
- Sie sind die Grundlage für alle Aktionen und Assertions im Test.
- Playwright priorisiert benutzerorientierte Locators (z.B. sichtbarer Text, Rollen) für robuste Tests.

User-Facing Locators

User-Facing Locators sind Locators, die sich an den sichtbaren und zugänglichen Eigenschaften einer Benutzeroberfläche orientieren – also an dem, was echte Nutzer:innen sehen und womit sie interagieren. Dazu zählen beispielsweise sichtbarer Text, Rollen (wie `button`, `link`), Labels von Formularfeldern, Platzhaltertexte, Alt-Texte von Bildern oder Titel-Attribute.

Warum User-Facing Locators verwenden?

- **Robustheit:** Sie sind weniger anfällig für Änderungen im Code, da sie sich auf stabile, für Nutzer:innen sichtbare Eigenschaften stützen – nicht auf interne Implementierungsdetails wie CSS-Klassen oder verschachtelte Strukturen.
- **Lesbarkeit:** Tests mit User-Facing Locators sind verständlicher, da sie die tatsächliche Nutzerinteraktion widerspiegeln.
- **Barrierefreiheit:** Sie fördern gute Accessibility, da sie auf zugängliche Attribute wie Rollen und Labels setzen.
- **Wartbarkeit:** Änderungen am Styling oder an der Struktur der Seite führen seltener zu fehlschlagenden Tests, solange die Benutzeroberfläche gleich bleibt.

Beispiele für User-Facing Locators in Playwright

```
1 // Nach sichtbarem Text
2 await Page.GetByText("Anmelden").ClickAsync();
3
4 // Nach Rolle (z.B. Button)
5 await Page.GetByRole(AriaRole.Button, new() { Name = "Absenden" }).ClickAsync();
6
7 // Nach Label (z.B. für Inputs)
8 await Page.GetByLabel("Benutzername").FillAsync("user123");
9
10 // Nach Platzhaltertext
11 await Page.GetByPlaceholder("Passwort eingeben").FillAsync("pass123");
12
13 // Nach Alt-Text (z.B. für Bilder)
14 await Page.GetByAltText("Firmenlogo").IsVisibleAsync();
```

Fazit:

User-Facing Locators machen Tests stabiler, verständlicher und nachhaltiger. Sie sollten immer bevorzugt werden – Test-IDs oder CSS/XPath-Locators sind nur dann sinnvoll, wenn keine benutzerorientierten Locators möglich sind.

Alternative Locators in Playwright

Neben den benutzerorientierten Locators (wie `GetByRole`, `GetByText`, `GetByLabel` usw.) bietet Playwright auch **alternative Locators** für spezielle Anwendungsfälle oder wenn User-Facing-Locators nicht ausreichen.

1. CSS-Locators

- Verwende Standard-CSS-Locators, um Elemente gezielt anzusprechen.
- Beispiel:

```
await Page.Locator(".my-class > input[type='email']").FillAsync("test@example.com");
```

2. XPath-Locators

- Beispiel:

```
await Page.Locator("//button[@type='submit']").ClickAsync();
```

3. Test-IDs

- Mit expliziten Attributen wie `data-testid` lassen sich stabile Locators definieren.
- Beispiel:

```
await Page.GetByTestId("checkout-payment-submit").ClickAsync();
```

4. Kombination und Filter

- Playwright erlaubt das Kombinieren und Filtern von Locators:

```
await Page.Locator("button").Filter(new() { HasText = "Speichern" }).ClickAsync();
```

Hinweis:

Alternative Locators sind nützlich, wenn keine benutzerorientierten Locators möglich sind. Sie sollten jedoch mit Bedacht eingesetzt werden, da sie oft weniger robust gegenüber UI-Änderungen sind.

Locators-Strategien und Best Practices

Locators: Empfohlene Reihenfolge

1. User-facing Attribute (Was der Benutzer sieht/interagiert):

- `Page.GetByRole()` (Zugänglichkeit: Buttons, Links, Headings etc.)
- `Page.GetByText()` (Sichtbarer Text)
- `Page.GetByLabel()` (Verknüpfte Formular-Labels)
- `Page.GetByPlaceholder()` (Platzhaltertext in Inputs)
- `Page.GetByAltText()` (Alt-Text von Bildern)
- `Page.GetByTitle()` (Titel-Attribut)

➡ **Warum diese? Sie sind robust, lesbar & fördern barrierefreie Anwendungen!**

2. Test-spezifische Attribute (Wenn nichts anderes geht):

- `Page.GetByTestId()` (Explizite `data-testid` o.ä. für Tests)

💡 **Wie finde ich diese? Chrome DevTools (Accessibility Tab) & Playwright Inspector/Codegen sind deine Werkzeuge! (Mehr dazu gleich!)**

Locators: Zu vermeiden (wenn möglich)

- CSS-Locators, die auf Implementierungsdetails basieren (Styling-Klassen, komplexe Strukturen)
 - Beispiel: `.button-styles.primary > span`
- XPath-Locators (oft komplex und weniger lesbar)
- Klassen/IDs, die sich ändern könnten (z.B. durch CSS-Module generiert)
- Indizes ohne klare semantische Bedeutung (z.B. `li:nth-child(3)`)

Tipp

Verwende den Codegen (``pwsh bin/Debug/net9.0/playwright.ps1 codegen``) oder den Inspector (``PWDEBUG=1 dotnet test``), um gute, resiliente Locators zu finden! Playwright priorisiert automatisch benutzerorientierte Locators.

Test-IDs – Best Practices

- **Konsistente Benennung:**
 - Verwende ein klares Namensschema
 - z.B. `feature-component-action`
 - Beispiel: `checkout-payment-submit`
- **Strukturierte Hierarchie:**
 - Eltern-Kind-Beziehungen abbilden
 - z.B. `user-menu` und `user-menu-profile`
- **Vermeidung von Duplikaten:**
 - Eindeutige IDs im gesamten Projekt
 - Automatisierte Prüfung auf Duplikate
- **Dokumentation:**
 - Test-IDs in Komponenten-Dokumentation aufnehmen

Wann Test-IDs verwenden?

Test-IDs: Vorteile

- **Stabile Locators:** Unabhängig von Textänderungen (Übersetzungen) oder UI-Strukturänderungen.
- **Explizit für Tests:** Macht klar, dass ein Element für die Automatisierung wichtig ist.
- **Entkopplung:** Trennt Testlogik von der genauen UI-Implementierung.
- **Klar dokumentierte Testbarkeit:** Signalisiert, welche Teile der UI testbar sein sollen.

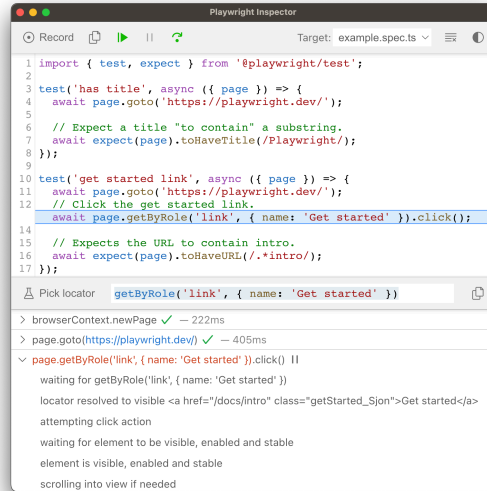
Test-IDs: Nachteile & Empfehlung

- **Zusätzlicher Markup-Overhead:** Müssen im Code hinzugefügt und gepflegt werden.
- **Nicht benutzerorientiert:** Testen nicht, wie ein echter Benutzer die Seite wahrnimmt (z.B. über sichtbaren Text oder Rollen).
- **Kann zu "Test-only"-Denken führen:** Elemente könnten nur für Tests hinzugefügt werden, ohne echten Nutzen.
- **Nicht für Barrierefreiheit relevant:** Im Gegensatz zu Rollen-Locators.

Empfehlung

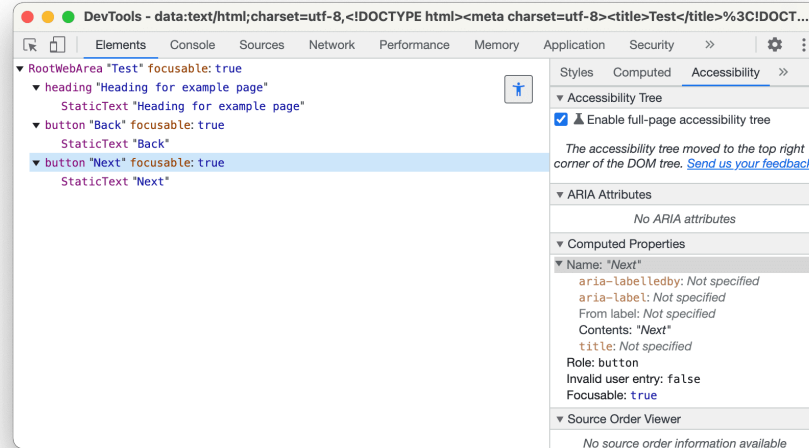
Verwende Test-IDs als ****Fallback****, wenn benutzerorientierte Locators (Rolle, Text, Label etc.) nicht eindeutig, stabil oder praktikabel sind. Priorisiere immer benutzerorientierte Locators!

Locators finden mit Playwright Inspector



- **Starten:** `dotnet test -- Playwright.LaunchOptions.Headless=false` oder `PWDEBUG=1 dotnet test`.
- **Inspector:** Interaktives Auswählen von Elementen im Browser.
- **Locator-Vorschläge:** Playwright schlägt automatisch gute Locators vor.
- **Testen & Verfeinern:** Locators direkt im Inspector ausprobieren.

Locators finden mit Chrome DevTools



Die Chrome DevTools bieten im "Accessibility"-Tab die Möglichkeit, den sogenannten **Accessibility Tree** einer Webseite einzusehen. Dieser Baum zeigt, wie Screenreader und andere Hilfsmittel die Seite wahrnehmen. Jedes Element im Accessibility Tree besitzt Eigenschaften wie **Rolle** (z.B. `button`, `link`), **Name** (sichtbarer Text), **Label**, **Beschreibung** und weitere zugängliche Attribute.

Locators finden mit Chrome DevTools

So nutzt du den Accessibility Tree:

1. Öffne die Chrome DevTools (`F12` oder Rechtsklick → "Untersuchen").
2. Wähle das gewünschte Element im "Elements"-Tab aus.
3. Wechsle zum Tab **"Accessibility"** (ggf. unter ">>" versteckt).
4. Sieh dir Rolle, Name, Label und weitere Eigenschaften an.

Tipp:

Wenn ein Element im Accessibility Tree nicht wie erwartet erscheint, kann das auf fehlende oder fehlerhafte ARIA-Attribute oder Labels hindeuten – ein wichtiger Hinweis für die Verbesserung der Barrierefreiheit und für stabile Playwright-Locators!

Locators Filter

- **Filter nach Text:** `.Filter(new() { HasText = "Suchtext" })`
- **Filter nach Kindelementen:** `.Filter(new() { Has = Page.GetByRole(AriaRole.Button) })`
- **Filter nach Sichtbarkeit:** `.Filter(new() { Visible = true })`
- **Filter nach Position:** `.First()` , `.Last()` , `.Nth(index)`
- **Kombinierte Filter:** Mehrere Filter können verkettet werden
- **Strikte Filterung:** Playwright wirft einen Fehler, wenn mehrere Elemente gefunden werden

Hinweis

Vermeide `.First()`, `.Last()` und `.Nth()` wenn möglich. Besser: Eindeutige Locators verwenden!

Locators Filter - Beispiele

```
1 // Nach Text filtern
2 await Page.Locator("li").Filter(new() { HasText = "News" }).ClickAsync();
3
4 // Nach Kindelement filtern
5 await Page.Locator("article").Filter(new() { Has = Page.GetByRole(AriaRole.Button, new() { Name = "Mehr" }) }).First.ClickAsync();
6
7 // Nur sichtbare Elemente auswählen
8 var visibleItems = Page.Locator(".menu-item").Filter(new() { HasText = "Aktiv" });
9 await Expect(visibleItems).ToHaveCountAsync(1);
10
11 // Erstes, letztes oder bestimmtes Element auswählen
12 await Page.Locator(".list-item").First.ClickAsync();
13 await Page.Locator(".list-item").Last.ClickAsync();
14 await Page.Locator(".list-item").Nth(2).ClickAsync(); // drittes Element (0-basiert)
15
16 // Kombinierte Filter: Nach Text und Kindelement
17 await Page.Locator("section").Filter(new() { HasText = "Wichtig", Has = Page.GetByRole(AriaRole.Button, new() { Name = "Details" }) });
18
19 // Strikte Filterung: Fehler, wenn mehrere Elemente gefunden werden
20 await Page.GetByRole(AriaRole.Button, new() { Name = "Löschen" }).ClickAsync(); // schlägt fehl, wenn mehr als ein Button mit diese
```

Locators Übung

Ziel: Lerne die verschiedenen Locators-Methoden von Playwright kennen.

Aufgaben:

1. Öffne die Playwright Dokumentation: <https://playwright.dev/java/>
2. Erstelle einen neuen Test (`Tests/SelectorsTest.cs`)
3. Finde passende Locators für folgende Elemente:
 - Die Hauptüberschrift der Seite
 - Den "Docs" Link im Header
 - Das Suchfeld in der Navigation
 - Den Dark/Light Mode Toggle Button
4. Nutze dabei bevorzugt User-Facing Locators wie `GetByRole()` , `GetByText()` etc.
5. Führe den Test aus und prüfe, ob alle Elemente korrekt gefunden werden

Tipp: Nutze den Playwright Inspector, um die passenden Locators zu finden!

Playwright Actions

Was sind Playwright Actions?

- Aktionen, mit denen Playwright Benutzerinteraktionen imitiert
- Beispiele: Klicken, Tippen, Ausfüllen von Formularen, Auswählen, Drag & Drop
- Simulieren echtes Nutzerverhalten im Browser
- Können mit Maus, Tastatur oder Touch ausgeführt werden
- Grundlage für alle Interaktionen und Testszenarien
- Unterstützen Auto-Waiting für stabile Tests
- Ermöglichen komplexe Abläufe wie Datei-Uploads oder Mehrfachauswahl

Aktionen in Playwright

```
1 // Grundlegende Aktionen
2 await Page.GotoAsync("https://example.com");
3 await Page.GetByRole(AriaRole.Button).ClickAsync();
4 await Page.GetByLabel("Passwort").FillAsync("12345");
5 await Page.GetByRole(AriaRole.Checkbox).CheckAsync();
6
7 // Erweiterte Aktionen
8 await Page.Keyboard.PressAsync("Enter");
9 await Page.Mouse.MoveAsync(100, 200);
10 await Page.GetByText("Über uns").HoverAsync();
11 await Page.GetByLabel("Datei").SetInputFilesAsync("upload.jpg");
```

Erweiterte Interaktionen

```
1 // Drag & Drop
2 await Page.GetByTestId("draggable").DragToAsync(Page.GetByTestId("target"));
3
4 // Mehrere Tasten gleichzeitig drücken
5 await Page.Keyboard.PressAsync("Control+A");
6 await Page.Keyboard.PressAsync("Control+C");
7
8 // Mausaktionen
9 await Page.Mouse.DownAsync();
10 await Page.Mouse.MoveAsync(100, 200);
11 await Page.Mouse.UpAsync();
12
13 // Mehrfachklick
14 await Page.GetByText("Doppelklick").DbClickAsync();
15
16 // Rechtsklick
17 await Page.GetByText("Kontextmenü").ClickAsync(new() { Button = MouseButton.Right });
18
19 // Mit Modifiern klicken
20 await Page.GetByRole(AriaRole.Link).ClickAsync(new() { Modifiers = new[] { KeyboardModifier.Shift } });
```

Formular-Interaktionen

```
1 // Formular-Interaktionen
2 await Page.GetByLabel("Benutzername").FillAsync("user123");
3 await Page.GetByLabel("Passwort").FillAsync("password123");
4 await Page.GetByLabel("E-Mail").FillAsync("user@example.com");
5
6 // Checkbox und Radio-Buttons
7 await Page.GetByLabel("Nutzungsbedingungen akzeptieren").CheckAsync();
8 await Page.GetByLabel("Newsletter abonnieren").UncheckAsync();
9 await Page.GetByLabel("Option 2").CheckAsync(); // Radio-Button
10
11 // Dropdown-Menüs
12 await Page.GetByLabel("Land").SelectOptionAsync("Deutschland");
13 await Page.GetByLabel("Sprache").SelectOptionAsync(new SelectOptionValue { Label = "Deutsch" });
14 await Page.GetByLabel("Kategorie").SelectOptionAsync(new SelectOptionValue { Value = "tech" });
15
16 // Datei-Upload
17 await Page.GetByLabel("Profilbild").SetInputFilesAsync("path/to/image.jpg");
18 await Page.GetByLabel("Dokumente").SetInputFilesAsync(new[] { "doc1.pdf", "doc2.pdf" });
19
20 // Formular absenden
21 await Page.GetByLabel(AriaLabel.Button, new() { Name = "Absenden" }).ClickAsync();
```

Übung: Todo MVC App Interaktionen

Ziel: Verschiedene Benutzerinteraktionen mit der Todo MVC App testen.

Aufgaben:

1. Öffne die Todo MVC App <https://todomvc.com/examples/react/dist/>
2. Füge ein neues Todo-Element hinzu
3. Bearbeite ein bestehendes Todo-Element
4. Lösche ein Todo-Element

Tipps:

- Nutze die passenden Playwright-Aktionen für jede Interaktion
- Achte auf die korrekten Locators
- Prüfe nach jeder Aktion, ob sie erfolgreich war

Assertions

Assertions in Playwright – Was sind Assertions?

- **Eine Assertion prüft, ob ein erwarteter Zustand im Test erfüllt ist.** Sie ist die Grundlage für verlässliche und aussagekräftige Tests.

Beispiel: Erwartung, dass ein Button sichtbar ist oder ein Text angezeigt wird. Ein Test ohne Assertions prüft nichts und ist wertlos! - **Playwright bietet mächtige und flexible Assertions.**

Assertions in Playwright – Auto-Retrying Assertions

- **Auto-Retrying Assertions** warten automatisch auf den gewünschten Zustand
- Typische Beispiele:
 - `ToBeVisibleAsync`
 - `ToHaveTextAsync`
 - `ToHaveURLAsync`
 - `ToBeCheckedAsync`
- Reduziert flaky Tests, da Playwright mehrfach prüft, bis das Ergebnis stimmt oder ein Timeout erreicht ist
- Ideal für UI-Elemente, die asynchron erscheinen oder sich ändern

Assertions in Playwright – Auto-Retrying Assertions: Beispiele

```
1 // Sichtbarkeit prüfen
2 await Expect(Page.GetByRole(AriaRole.Button, new() { Name = "Login" })).ToBeVisibleAsync();
3
4 // Textinhalt prüfen
5 await Expect(Page.GetByTestId("headline")).ToHaveTextAsync("Willkommen!");
6
7 // URL prüfen
8 await Expect(Page).ToHaveURLAsync(new Regex(".*dashboard"));
9
10 // Checkbox angehakt?
11 await Expect(Page.GetByLabel("AGB akzeptieren")).ToBeCheckedAsync();
12
13 // Attribut prüfen
14 await Expect(Page.GetByRole(AriaRole.Image)).ToHaveAttributeAsync("alt", "Logo");
15
16 // Anzahl der Elemente prüfen
17 var items = Page.Locator(".todo-item");
18 await Expect(items).ToHaveCountAsync(3);
```

Assertions in Playwright – Non-Retrying Assertions

- **Non-Retrying (generische) Assertions prüfen synchron einen Wert, ohne zu warten**
- Typische Beispiele:
 - `ToBe`
 - `ToEqual`
 - `ToContain`
 - `ToBeTruthy`
- Typisch für Variablen, API-Responses, interne Logik

Assertions in Playwright – Non-Retrying Assertions: Beispiele

```
1 // Statuscode einer API-Response
2 Expect(response.Status).ToBe(200);
3
4 // Array enthält Wert
5 Expect(items).ToContain("News");
6
7 // Objektvergleich
8 Expect(user).ToEqual(new { id = 1, name = "Max" });
9
10 // Wahrheitswert
11 Expect(isLoggedIn).ToBeTruthy();
```

Assertions in Playwright – Soft Assertions

- Mit `Expect.Multiple()` können mehrere Fehler gesammelt werden, ohne den Test sofort abubrechen
- Am Ende des Tests werden alle Fehler gemeinsam gemeldet
- Nützlich, um mehrere Aspekte in einem Test zu prüfen

Assertions in Playwright – Soft Assertions: Beispiele

```
1 // Mehrere UI-Elemente prüfen, Fehler werden gesammelt
2 // Hinweis: In .NET Playwright werden Soft Assertions anders implementiert
3 // Verwende Multiple() für gesammelte Assertions:
4 await Expect.Multiple(() =>
5 {
6     Expect(Page.GetByText("A")).ToBeVisibleAsync();
7     Expect(Page.GetByText("B")).ToBeVisibleAsync();
8     Expect(Page.GetByText("C")).ToBeVisibleAsync();
9     Expect(apiResponse.Status).ToBe(200);
10 });
```

Assertions in Playwright – Best Practices

- **Nutze auto-retry Assertions für UI-Checks**
- Schreibe präzise, beschreibende Fehlermeldungen mit aussagekräftigen Assertions
- Assertions gehören in die Tests, nicht ins Page Object
- Vermeide unnötige `waitFor` -Aufrufe – Playwright wartet automatisch
- **Nutze `.Not` für Negationen:** `await Expect(locator).Not.ToBeVisibleAsync();`
- Halte Assertions so spezifisch wie möglich, um Fehlerursachen schnell zu erkennen

Auto-Wait und Synchronisation

- Playwright **wartet automatisch** auf Elemente
- Keine expliziten Waits notwendig
- Wartet auf verschiedene Bedingungen:
 - Element ist sichtbar
 - Element ist aktiviert
 - Element ist stabil (keine Animation)
 - Netzwerkanfragen sind abgeschlossen

```
1 // Kein explizites Warten notwendig
2 await Page.getByRole(AriaRole.Button, new() { Name = "Speichern" }).ClickAsync();
3 // Playwright wartet, bis der Button klickbar ist
```

Übung 1 – Erster Test

Ziel: Ersten Playwright-Test für die Feed App schreiben.

Aufgaben:

1. Test erstellen (`Tests/NavigationTest.cs`).
2. Startseite öffnen, auf "View Public News" klicken.
3. Prüfen, ob zur `/news/public` Seite navigiert wurde (URL, Überschrift).
4. Test ausführen (`dotnet test`).

Zeit: 20 Minuten

Nachbesprechung

Übung 2 – Filtertest im News Feed

Ziel: Filter- und Suchfunktionen im News Feed testen.

Aufgaben:

1. Test erstellen (z.B. `Tests/NewsFeedTest.cs`).
2. Zur `/news/public` Seite navigieren.
3. Suchfunktion testen (Begriff eingeben, Ergebnisse prüfen, leeren).
4. Kategorie-Filter testen (Kategorie wählen, Ergebnisse prüfen, zurücksetzen).
5. Trace aktivieren & im Report analysieren (`dotnet test -- Playwright.Trace=on`).

Zeit: 25 Minuten

Nachbesprechung

Test-Annotierungen in Playwright

In MSTest können Tests mit verschiedenen **Attributen** versehen werden, um sie gezielt zu steuern oder zu konfigurieren. Dies ist besonders nützlich, um Tests zu überspringen, zu kategorisieren oder zusätzliche Metadaten hinzuzufügen.

Arten von Annotierungen in MSTest

- `[TestCategory]` : Kategorisiert Tests für Filterung
- `[Ignore]` : Überspringt einen Test mit optionalem Grund
- `[Timeout]` : Setzt maximale Ausführungszeit für Tests
- `[DataRow]` : Ermöglicht parameterisierte Tests
- `[Priority]` : Setzt Ausführungspriorität für Tests
- `[TestContext]` : Zugriff auf Test-Metadaten und -Kontext

Beispiele für Annotierungen

```
1 // Test überspringen
2 using Microsoft.Playwright.MSTest;
3
4 [TestClass]
5 public class AnnotationTests : PageTest
6 {
7     [TestMethod]
8     [Ignore("Feature noch nicht implementiert")]
9     public async Task DieserTestWirdÜbersprungen()
10    {
11        await Page.GotoAsync("https://example.com");
12    }
13
14    // Test nur in Chromium ausführen
15    [TestMethod]
16    [TestCategory("Chromium")]
17    public async Task NurInChromium()
18    {
19        if (BrowserName != "chromium")
20            Assert.Inconclusive("Nur für Chromium relevant");
21    }
22 }
```

Beispiele für Test-Steps

TestMethod

```
1  public async Task TestMitSteps()
2  {
3      // In MSTest können Test-Steps durch strukturierten Test-Code ersetzt werden
4      // oder durch TestContext-Logging für bessere Nachverfolgbarkeit
5
6      TestContext.WriteLine("Step: Log in");
7      await LoginAsync();
8
9      TestContext.WriteLine("Step: Outer step");
10     await PerformOuterStepAsync();
11
12     TestContext.WriteLine("Step: Inner step");
13     await PerformInnerStepAsync();
14 }
15
16 private async Task LoginAsync()
17 {
18     // Login-Logik
19 }
20
21 private async Task PerformOuterStepAsync()
```

Best Practices für Annotierungen

- Verwende Annotierungen sparsam, um Tests nicht unnötig zu fragmentieren.
- Dokumentiere den Grund für jede Annotierung, um die Nachvollziehbarkeit zu gewährleisten.
- Nutze `[TestCategory]` für bessere Organisation und Filterung von Tests.
- Verwende `[Ignore]` mit aussagekräftigen Begründungen für temporär deaktivierte Tests.

Teststruktur im Detail (Fortsetzung)

```
1  // Weiterer Test in der Gruppe
2  [TestMethod]
3  public async Task ZeigtRichtigenInhalt()
4  {
5      var heading = Page.Locator("h1");
6      await Expect(heading).ToHaveTextAsync("Example Domain");
7      var paragraph = Page.Locator("p").First;
8      await Expect(paragraph).ToContainTextAsync("for illustrative examples");
9  }
10
11 // Test mit Bedingung überspringen
12 [TestMethod]
13 public async Task ZeigtBenutzermenü()
14 {
15     // Test nur in Chrome ausführen
16     if (BrowserName != "chromium")
17         Assert.Inconclusive("Nur in Chrome getestet");
18
19     // Testlogik...
20 }
```


Test-Hooks und Fixtures

TestClass

```
1  public class ProduktSeitenTests : BrowserTest
2  {
3      // Vor allen Tests in dieser Gruppe ausführen
4      [ClassInitialize]
5      public static async Task ClassInitialize(TestContext context)
6      {
7          // Gemeinsames Setup, z.B. Testdaten erstellen
8          // Für Browser-bezogene Operationen nutze TestInitialize
9      }
10
11     // Nach allen Tests in dieser Gruppe ausführen
12     [ClassCleanup]
13     public static async Task ClassCleanup()
14     {
15         // Aufräumen, z.B. Testdaten löschen
16     }
17 }
```

Test-Hooks und Fixtures (Fortsetzung)

```
1  // Vor jedem Test ausführen
2  [TestInitialize]
3  public async Task TestInitialize()
4  {
5      await Page.GotoAsync("https://example.com/products");
6      // Weitere Setup-Schritte ...
7  }
8
9  // Nach jedem Test ausführen
10 [TestCleanup]
11 public async Task TestCleanup()
12 {
13     // Screenshots erstellen
14     await Page.ScreenshotAsync(new() { Path = "test-results/screenshot.png" });
15     // Logs sammeln, etc.
16 }
17
18 // Einzelne Tests ...
19 [TestMethod]
20 public async Task ProduktIstSichtbar()
21 {
```

Playwright Debugging

Tests verstehen und Fehler finden



Daniel Sogl - linktr.ee/daniel_sogl

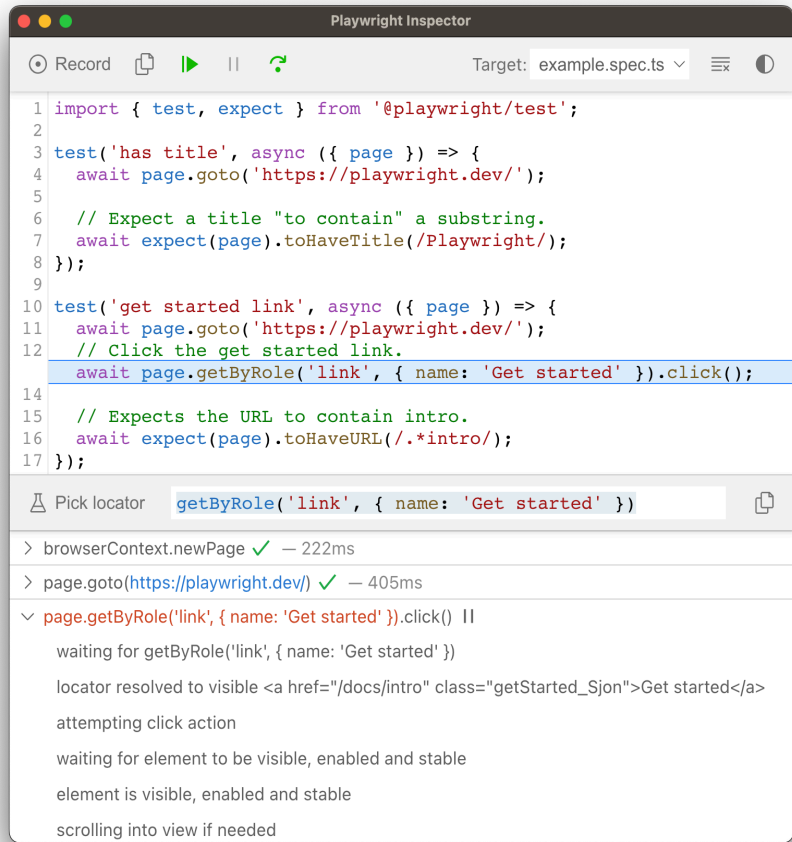
Debugging mit Playwright Inspector

- **Visuelle Debugging-Oberfläche**
- **Step-by-Step Ausführung:** Durch Tests navigieren
- **DOM-Inspektor:** Elemente untersuchen und Locators testen
- **Konsole:** JavaScript im Browserkontext ausführen
- **Breakpoints:** Test an bestimmten Stellen anhalten

```
1  # Mit Inspector starten
2  PWDEBUG=1 dotnet test
3
4  # UI-Modus für interaktives Debugging
5  dotnet test --logger trx --results-directory TestResults
```

Playwright Inspector im Detail

- **Action Explorer:** Zeigt alle Aktionen im Test
- **Watch Mode:** Automatische Aktualisierung bei Codeänderungen
- **Selector Explorer:** Hilft bei der Auswahl von Locators
- **Network Monitor:** Zeigt Netzwerkanfragen und -antworten
- **Console:** Zeigt Konsolenausgaben und Fehler
- **Source Panel:** Zeigt den Testcode mit Breakpoints



Debugging-Strategien - Entwicklung

Während der Entwicklung

- `PWDEBUG=1` Umgebungsvariable für interaktives Debugging
- `--logger trx` für detaillierte Testergebnisse
- **Headed-Modus** in PlaywrightConfig konfigurieren
- `Page.PauseAsync()` für manuelle Breakpoints
- `Console.WriteLine()` für Variableninspektion

Bei fehlgeschlagenen Tests

- **Trace-Dateien** analysieren
- **Screenshots und Videos** prüfen
- **Netzwerkanfragen** untersuchen
- **Konsolenausgaben** überprüfen
- **Locators** mit Inspector testen

Debugging-Tipps im Code

Debugging-Tipps im Code

```
1 // Debugging-Tipps im Code
2 [TestMethod]
3 public async Task DebuggingBeispiel()
4 {
5     // Testausführung pausieren
6     await Page.PauseAsync();
7
8     // Variablen in der Konsole ausgeben
9     var url = Page.Url;
10    Console.WriteLine($"Aktuelle URL: {url}");
11
12    // DOM-Zustand in der Konsole ausgeben
13    await Page.EvaluateAsync(@"() => {
14        console.log('DOM-Struktur:', document.body.innerHTML);
15    }");
16
17    // Locators testen
18    var buttonExists = await Page
19        .GetByRole(AriaRole.Button, new() { Name = "Submit" })
20        .IsVisibleAsync();
21    Console.WriteLine($"Button sichtbar: {buttonExists}");
```


Debugging – Einfache Techniken

Einfaches Debugging

```
1 // Testausführung pausieren
2 await Page.PauseAsync();
3
4 // Screenshot erstellen
5 await Page.ScreenshotAsync(new()
6 {
7     Path = "debug.png"
8 });
9
10 // Konsole nutzen
11 Console.WriteLine($"Current URL: {Page.Url}");
```

Mit Trace Viewer

```
1 // In .runsettings
2 <TestRunParameters>
3     <Parameter name="Trace" value="on-first-retry" />
4 </TestRunParameters>
5
6 // Trace für einen Test starten
7 await Context.Tracing.StartAsync(new() { Screenshots = true
8
9 // ... Test ausführen ...
10
11 // Trace speichern
12 await Context.Tracing.StopAsync(new() { Path = "trace.zip" }
```

Trace Viewer – Analyse fehlgeschlagener Tests

Playwright demo-todo-app.spec.ts:298 > Clear completed button > should remove completed items when clicked

0 500ms 1.0s 1.5s 2.0s 2.5s 3.0s 3.5s 4.0s 4.5s 5.0s 5.5s

Actions Metadata Action Before After

> Before Hooks 389ms

locator.check `getByTestId('t...` 20ms

locator.click `getByRole('but...` 66ms

expect.toHaveCount `getByTe...` 5.0s

> After Hooks 0ms

480ms expect.toHaveCount `getByTestId('todo-item')` 5.0s

Timed out 5000ms waiting for `expect(received).toHaveLength(expected) // deep equality`

Expected: 3
Received: 2

Call log:

- expect.toHaveCount with timeout 5000ms
- waiting for `getByTestId('todo-item')`
- locator resolved to 2 elements
- unexpected value "2"

Trace-Funktionen im Detail

Was wird aufgezeichnet

- Screenshots (vor/nach jeder Aktion)
- DOM-Snapshots
- Netzwerkanfragen
- Konsolenausgaben
- Testschritte mit Zeitstempeln
- Browserkontext-Informationen

Konfigurationsoptionen

```
1  <!-- In .runsettings -->
2  <TestRunParameters>
3    <Parameter name="Trace" value="on-first-retry" />
4    <!-- Alternativen:
5      'on' - immer aufzeichnen
6      'off' - nie aufzeichnen
7      'retain-on-failure' - nur bei Fehlern behalten -->
8  </TestRunParameters>
```

Trace-Datei öffnen

```
1  # Trace-Datei direkt öffnen
2  pwsh bin/Debug/net8.0/playwright.ps1 show-trace trace.zip
3
4  # Oder über Browser
5  npx playwright show-trace trace.zip
```

- **Automatisch:** Browser öffnet sich mit Trace Viewer
- **Interaktiv:** Navigation durch alle Testschritte
- **Offline:** Funktioniert ohne Internetverbindung

Navigation im Trace Viewer

- **Timeline:** Chronologische Ansicht aller Aktionen
- **Actions:** Details zu jeder Playwright-Aktion
- **Source:** Zugehöriger Testcode mit Zeilennummern
- **Network:** HTTP-Requests mit Status und Timing
- **Console:** Browser-Console-Ausgaben

💡 **Tipp:** Klicke auf beliebige Aktion in der Timeline, um den entsprechenden Testcode und Screenshot zu sehen.

Praktische Trace-Analyse

TestMethod

```
1  public async Task TraceExample()  
2  {  
3      // Trace konfigurieren  
4      await Context.Tracing.StartAsync(new() {  
5          Screenshots = true,  
6          Snapshots = true,  
7          Sources = true  
8      });  
9  
10     // Test durchführen  
11     await Page.GotoAsync("https://example.com");  
12     await Page.ClickAsync("button");  
13  
14     // Bei Fehler: Trace automatisch speichern  
15     await Context.Tracing.StopAsync(new() {  
16         Path = $"trace-{TestContext.TestName}.zip"  
17     });  
18 }
```

Trace-Konfiguration in .runsettings

```
1  <TestRunParameters>
2    <!-- Immer Trace aufzeichnen -->
3    <Parameter name="Trace" value="on" />
4
5    <!-- Nur bei Fehlern behalten -->
6    <Parameter name="Trace" value="retain-on-failure" />
7
8    <!-- Bei erstem Retry -->
9    <Parameter name="Trace" value="on-first-retry" />
10
11   <!-- Nie aufzeichnen -->
12   <Parameter name="Trace" value="off" />
13 </TestRunParameters>
```

- `on` : Trace wird immer aufgezeichnet (Performance-Impact!)
- `retain-on-failure` : Trace nur bei fehlgeschlagenen Tests behalten
- `on-first-retry` : Empfohlene Einstellung für CI/CD

Global Setup für Traces

TestClass

```
1  public class TraceSetup
2  {
3      [AssemblyInitialize]
4      public static void Setup(TestContext context)
5      {
6          Environment.SetEnvironmentVariable("TRACE", "on");
7      }
8  }
```

- **Assembly-Level:** Gilt für alle Tests in der Assembly
- **Umgebungsvariable:** Überschreibt .runsettings
- **Flexibel:** Kann basierend auf Build-Konfiguration gesetzt werden

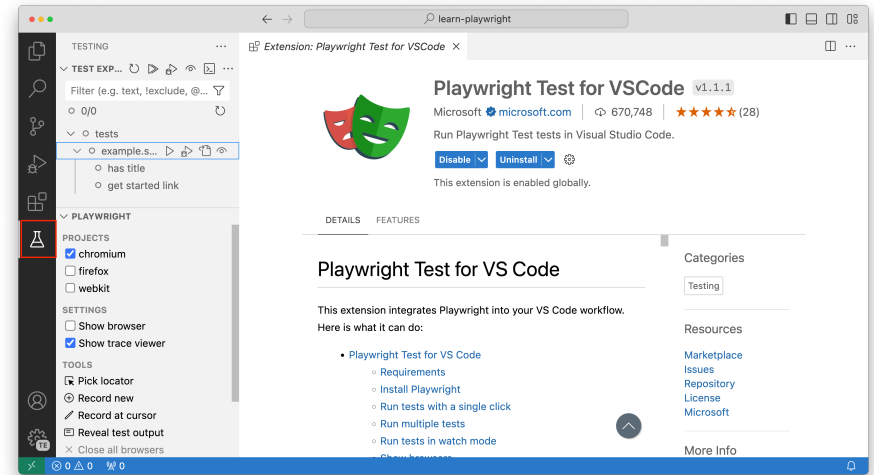
Programmatische Trace-Steuerung

TestMethod

```
1  public async Task AdvancedTraceExample()
2  {
3      // Conditional tracing basierend auf Testname
4      var shouldTrace = TestContext.TestName.Contains("Critical");
5
6      if (shouldTrace)
7      {
8          await Context.Tracing.StartAsync(new() {
9              Screenshots = true,
10             Snapshots = true,
11             Sources = true,
12             Title = $"Trace-{TestContext.TestName}",
13             Name = TestContext.TestName
14         });
15     }
16
17     try
18     {
19         // Test logic here
20         await Page.GotoAsync("https://example.com");
21         // more test steps
```

Playwright in der IDE: Visual Studio Code

- Offizielle Extension: **Playwright Test for VS Code** (ID: `ms-playwright.playwright`)
- **Test-Explorer**: Tests ausführen, debuggen und Ergebnisse direkt in der IDE sehen
- **Debugging**: Breakpoints setzen und interaktiv debuggen
- **Locator-Tools**:
 - Locators testen und inspizieren
 - Locators generieren ("Pick Locator")
- **Tests aufzeichnen** (Record new tests)
- **Trace-Viewer** und Screenshots direkt öffnen
- **Installation**: Im Extensions Marketplace nach "Playwright Test" suchen (oder ID: `ms-`



Playwright in der IDE: JetBrains (WebStorm, IntelliJ, ...)

- Native Playwright-Unterstützung ab WebStorm 2022.3
- Test-Explorer: Übersicht und Ausführung aller Tests
- Debugging mit Breakpoints und Step-by-Step
- Trace-Viewer Integration
- Automatische Erkennung von Playwright-Tests nach Installation als NuGet-Package
- Keine zusätzliche Extension notwendig

Authentifizierung & Teststrukturen

Tag 2: Auth-Handling, Fixtures & Page Objects



Wiederholung: Tag 1 Konzepte

- **Grundlagen von Playwright:**
 - Installation und Konfiguration
 - Locators und Aktionen
 - Assertions und Auto-Wait
- **Wichtige Erkenntnisse:**
 - Locators sollten robust und semantisch sein
 - Auto-Wait reduziert flaky Tests
 - Teststruktur beeinflusst Wartbarkeit
- **Offene Fragen aus Tag 1?**

Tagesagenda im Detail

- **Vormittag:**
 - Accessibility Testing
 - Komplexe UI-Interaktionen
 - Authentifizierung (Strategien, Storage State)
 - Session-Handling
- **Nachmittag:**
 - Test-Strukturierung vor dem POM
 - Fixtures (Eingebaute, Eigene)
 - Page Object Model (Einführung, Architektur, Best Practices)
 - Praktische Übungen (Login, Fixtures, POM Refactoring)
- **Offene Fragen aus Tag 1?**

Accessibility Testing mit Playwright

- **Barrierefreiheit (Accessibility, a11y):** Digitale Produkte für alle Menschen nutzbar machen
- **Warum testen?**
 - Gesetzliche Vorgaben (z.B. BITV, WCAG)
 - Bessere Usability für alle
 - Frühzeitiges Erkennen von Problemen spart Kosten
- **Automatisierte Tests:** Finden viele, aber nicht alle Probleme

Warum Accessibility wichtig ist

- **Gesetzliche Anforderungen:**
 - BITV 2.0 (Barrierefreie-Informationstechnik-Verordnung)
 - Barrierefreiheitsstärkungsgesetz (BFSG) ab 28. Juni 2025
 - Internationale WCAG-Standards
- **Wirtschaftliche Vorteile:**
 - Erweiterung der Zielgruppe
 - Verbesserte SEO durch semantische Struktur
 - Höhere Kundenzufriedenheit und -bindung
- **Technische Benefits:**
 - Robustere und wartbarere Code-Struktur
 - Bessere Testbarkeit durch semantische Locators

Accessibility-Checks mit Playwright & axe-core

- **Deque.AxeCore.Playwright:** Integration des beliebten axe Accessibility-Scanners
- **Typische Checks:**
 - Fehlende Labels, schlechte Kontraste, doppelte IDs, etc.
- **Integration:**
 - `dotnet add package Deque.AxeCore.Playwright`
 - Im Test importieren und verwenden
- **Bonus:** Erkenntnisse aus Accessibility-Tools (wie `axe-core` oder Chrome DevTools) helfen nicht nur Barrierefreiheitsprobleme zu beheben, sondern auch die **besten und robustesten Playwright-Locators** zu finden!

Accessibility Regeln

- **WCAG 2.0, 2.1, 2.2 Konformität**
 - Level A (Basis-Anforderungen)
 - Level AA (Wichtige Verbesserungen)
 - Level AAA (Optimale Zugänglichkeit)
- **Häufige Testfälle:**
 - Fehlende Alt-Texte für Bilder
 - Unzureichende Farbkontraste
 - Fehlende ARIA-Labels
 - Fehlende Überschriften-Hierarchie
 - Tastatur-Navigation
 - Formular-Labels und Fehlermeldungen

Beispiel: Accessibility-Test

```
1  using Microsoft.Playwright;
2  using Microsoft.Playwright.MSTest;
3  using Deque.AxeCore.Playwright;
4  using Deque.AxeCore.Commons;
5
6  [TestClass]
7  public class AccessibilityTests : PageTest
8  {
9      [TestMethod]
10     public async Task SeiteIstBarrierefrei()
11     {
12         await Page.GotoAsync("https://example.com");
13         AxeResult results = await Page.RunAxe();
14
15         // Option 1: Strikt - Test schlägt bei Violations fehl
16         Assert.IsTrue(results.Violations == null || results.Violations.Length == 0,
17             $"Accessibility violations found: {results.Violations?.Length}");
18     }
19
20     [TestMethod]
21     public async Task SeiteErfulltWCAG21AStandards()
```

DevTools Recap: A11y & Locators

Erinnerung an Tag 1:

- Der "**Accessibility**"-Tab in den Chrome DevTools ist entscheidend.
- Er zeigt den **Accessible Name** und die **ARIA-Rolle** eines Elements.
- Diese Informationen sind die Basis für Playwrights User-Faced Selectors:
 - `GetByRole()`
 - `GetByLabel()`
 - `GetByText()`
 - `GetByPlaceholder()`
 - `GetByAltText()`
 - `GetByTitle()`

Übung 2A – Accessibility-Check mit Playwright

Ziel: Automatisierten Accessibility-Check für eine Seite schreiben.

Aufgaben:

1. **Installiere** `Deque.AxeCore.Playwright` **NuGet Package** (`dotnet add package Deque.AxeCore.Playwright`).
2. **Schreibe einen Playwright-Test**, der eine Seite aufruft und mit `Page.RunAxe()` prüft, ob keine **Accessibility-Violations** gefunden werden.
3. **Verwende** `AxeResult` **als Rückgabebetyp** und prüfe das `Violations` **Array**.
4. **Führe den Test aus** und analysiere ggf. gefundene Probleme.

Zeit: 20 Minuten

Komplexe UI-Interaktionen

Vertiefung: Komplexe UI-Interaktionen

- **Web-Apps sind komplexer geworden:**
 - Dynamische Inhalte, die erst bei bestimmten Interaktionen sichtbar werden
 - Verschachtelte und komplexe Formularelemente
 - Hover-Menüs, Tooltips, Dropdowns, Dialoge
 - Tabellen mit sortierbaren Spalten und editierbaren Zellen
 - Drag & Drop, Slider, Multi-Select-Listen
- **Diese Interaktionen erfordern spezielle Techniken:**
 - Gezielte Hover-Aktionen für Interaktionen mit dynamisch erscheinenden Elementen
 - Präzise Tastatur-Events für Shortcuts oder spezielle Eingaben
 - Explizite Wartestrategien, wenn Auto-Wait nicht ausreicht

Beispiel: Hover und dynamisch erscheinende Elemente

TestMethod

```
1  public async Task LoeschenEinesTodosMitHoverAktion()
2  {
3      await Page.GotoAsync("https://demo.playwright.dev/todomvc/");
4      await Page.GetByPlaceholder("What needs to be done?").FillAsync("Hover-Test");
5      await Page.Keyboard.PressAsync("Enter");
6
7      // Prüfen, dass Item existiert
8      await Expect(Page.GetByText("Hover-Test")).ToBeVisibleAsync();
9
10     // Hover über dem Element, um den Lösch-Button sichtbar zu machen
11     await Page.GetByText("Hover-Test").HoverAsync();
12
13     // Klick auf den Lösch-Button (der erst nach Hover sichtbar ist)
14     // In TodoMVC ist der Lösch-Button ein Element mit der Klasse "destroy"
15     await Page.Locator(".todo-list li").Filter(new() { HasText = "Hover-Test" })
16         .GetByRole(AriaRole.Button, new() { Name = "Delete" })
17         .ClickAsync();
18
19     // Prüfen, dass Item gelöscht wurde
```


Beispiel: Tabellen-Interaktionen

TestMethod

```
1  public async Task Tabelleninteraktionen()
2  {
3      await Page.GotoAsync("https://demoqa.com/webtables");
4
5      // Suche eine bestimmte Zelle in der Tabelle mit Hilfe von Rollen-Lokatoren
6      var emailCell = Page.GetByRole(AriaRole.Gridcell, new() { Name = "alden@example.com" });
7      await Expect(emailCell).ToBeVisibleAsync();
8
9      // Bearbeite einen Benutzer durch Klicken des Bearbeiten-Buttons in einer bestimmten Zeile
10     await Page
11         .GetByRole(AriaRole.Row, new() { NameRegex = new Regex("alden@example.com") })
12         .Locator("[id^=\"edit-record\"]")
13         .ClickAsync();
14
15     await Page.Locator("[id^=\"edit-record\"]").First.ClickAsync();
16
17     // Fülle das Formular mit neuen Daten aus
18     await Page
19         .GetByRole(AriaRole.Textbox, new() { Name = "First Name" })
20         .FillAsync("Geänderter Name");
21     await Page.GetByRole(AriaRole.Button, new() { Name = "Submit" }).ClickAsync();
```

Beispiel: Tastatur-Interaktionen

```
1 // In vielen Web-Apps sind Tastaturkürzel wichtige Interaktionsmöglichkeiten
2 [TestMethod]
3 public async Task TastaturinteraktionenMitTodoItems()
4 {
5     // Setup: Todo-Item erstellen
6     await Page.GotoAsync("https://demo.playwright.dev/todomvc/");
7     await Page.GetByPlaceholder("What needs to be done?").FillAsync("Tastatur-Test");
8     await Page.Keyboard.PressAsync("Enter");
9
10    // Mit Tab zum Todo-Item navigieren
11    await Page.Keyboard.PressAsync("Tab");
12
13    // Mit Leertaste das Checkbox-Element aktivieren
14    await Page.Keyboard.PressAsync("Space");
15
16    // Bearbeiten eines Todos durch Doppelklick
17    var todoItem = Page.GetText("Tastatur-Test");
18    await todoItem.DblClickAsync();
19
20    // Text im Bearbeitungsmodus ändern
21    await Page.Keyboard.PressAsync("Control+A"); // Alles auswählen
```

Übung 2B – Komplexe Interaktionen mit TodoMVC

Ziel: Verschiedene fortgeschrittene Interaktionsstrategien mit der TodoMVC-App üben.

Demo-Website: <https://demo.playwright.dev/todomvc/>

Aufgaben:

1. Hover-Interaktion:

- Erstelle einen Test, der ein Todo-Item hinzufügt und es dann mit Hover + Klick auf das "x" löscht
- Erstelle ein weiteres Item und prüfe, dass der Lösch-Button erst nach Hover sichtbar wird

2. Tastatur-Interaktion:

- Erstelle einen Test, der ein Todo nur mit Tastatureingaben bearbeitet:
- Doppelklicke auf ein Todo und ändere es mit `Keyboard.PressAsync()` und `Keyboard.TypeAsync()`
- Markiere ein Todo als erledigt nur mit Tab-Navigation und Space-Taste

3. Filter-Interaktion:

- Erstelle Todos mit verschiedenen Status (aktiv/erledigt)

Session Handling

Authentifizierungsstrategien im Vergleich

- **UI-basierte Authentifizierung**
 - **Testet den vollständigen Login-Flow**
 - **Prüft UI-Elemente und Validierungen**
 - **Langsamer und fehleranfälliger**
 - **Realistischste Benutzererfahrung**
- **API-basierte Authentifizierung:**
 - **Direkter API-Aufruf zum Login**
 - **Umgeht UI-Interaktionen**
 - **Schneller und zuverlässiger**
 - **Weniger realistisch, aber effizienter**

Authentifizierungsmethoden

- **Token-basierte Auth:**
 - JWT (JSON Web Tokens)
 - Access & Refresh Tokens
 - Stateless Authentication
- **Session-basierte Auth:**
 - Cookie-basierte Sessions
 - Server-seitige Speicherung
 - Stateful Authentication
- **OAuth & OpenID Connect:**
 - Delegierte Authentifizierung
 - Single Sign-On (SSO)
 - Autorisierungscode-Flow

Herausforderungen beim Login Testing

- **Wiederholtes Login** in jedem Test ist ineffizient
- **Sitzungsdauer**: Tokens können ablaufen
- **UI-Login** ist langsam und fehleranfällig
- **Abhängigkeiten**: Tests hängen vom Auth-Provider ab
- **Sicherheit**: Testen mit echten Anmeldedaten

Tipp: Arbeiten mit Umgebungsvariablen

- **Umgebungsvariablen sind ideal für:**
 - **Base URLs** (`Environment.GetEnvironmentVariable("BASE_URL")`)
 - **Test-Zugangsdaten** (`Environment.GetEnvironmentVariable("TEST_USER")` , `Environment.GetEnvironmentVariable("TEST_PASSWORD")`)
 - **Konfigurationsunterschiede zwischen Umgebungen**
- **In** `runsettings.xml` **verwenden:**

```
<RunSettings>
  <Playwright>
    <BaseUrl>https://demo.playwright.dev/todomvc</BaseUrl>
  </Playwright>
</RunSettings>
```

- **In Tests verwenden:**

```
[TestMethod]
public async Task LoginMitEnvCredentials()
{
    await Page.GotoAsync("https://www.saucedemo.com/");
}
```


Authentifizierungsstrategien für Tests

Für E2E-Tests

- Storage State für wiederholte Tests
- API-basierte Auth für Effizienz
- Mock Auth Provider für Isolation
- Test-spezifische Benutzer

Für Komponententests

- Auth-Context mocken
- Direkte Zustandsmanipulation
- Geschützte Routen umgehen
- Berechtigungen simulieren

Sicherheitsaspekte beim Auth-Testing

- **Niemals produktive Credentials in Tests:**
 - Separate Testbenutzer verwenden
 - Testumgebung isolieren
- **Sichere Speicherung von Testdaten:**
 - Umgebungsvariablen für Credentials
 - Secrets-Management in CI/CD
- **Datenschutz beachten:**
 - Keine echten Benutzerdaten in Tests
 - Testdaten nach Testlauf bereinigen

Session-Handling mit Playwright – Theorie

- **Browser-Kontext:** Isolierte Browser-Umgebung
- **Storage State:** Speichern von Cookies und LocalStorage
- **Wiederverwendung:** State in neue Kontexte laden
- **Fixture-basiert:** Integration in Playwright's Testmodell

Storage State im Detail

```
1  // Struktur eines Storage State
2  {
3    "cookies": [
4      {
5        "name": "session",
6        "value": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... ",
7        "domain": "example.com",
8        "path": "/",
9        "expires": 1661201000,
10       "httpOnly": true,
11       "secure": true,
12       "sameSite": "Lax"
13     }
14   ],
15   "origins": [
16     {
17       "origin": "https://example.com",
18       "localStorage": [
19         {
20           "name": "token",
21           "value": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... "
```

UI-Auth Flow

```
1 // Login durchführen und Storage State speichern
2 using Microsoft.Playwright;
3 using Microsoft.Playwright.MSTest;
4
5 [TestClass]
6 public class AuthenticatedTests : PageTest
7 {
8     private static readonly string AuthStateFile = "playwright/.auth/user.json";
9
10    [TestMethod]
11    [Priority(1)] // Wichtig: Dieser Test sollte zuerst laufen
12    public async Task SetupAuthentication()
13    {
14        // Gehe zur Login-Seite
15        await Page.GotoAsync("http://localhost:3000/auth/signin");
16
17        // Führe Login durch
18        await Page.GetByLabel("Email").FillAsync("test@example.com");
19        await Page.GetByLabel("Password").FillAsync("password123");
20        await Page.GetByRole(AriaRole.Button, new() { Name = "Submit" }).ClickAsync();
21    }
```

API-basierte Authentifizierung

```
1 // Effizienter Login über API statt UI
2 using Microsoft.Playwright;
3 using Microsoft.Playwright.MSTest;
4
5 [TestClass]
6 public class ApiAuthTests : PageTest
7 {
8     private static readonly string AuthStateFile = "playwright/.auth/user.json";
9
10    [TestMethod]
11    [Priority(1)]
12    public async Task AuthenticateViaApi()
13    {
14        // API-Request für Login erstellen
15        var request = await Playwright.APIRequest.NewContextAsync();
16
17        var loginResponse = await request.PostAsync("http://localhost:3000/api/auth/signin", new()
18        {
19            DataObject = new
20            {
21                email = "test@example.com"
```

Storage State einbinden

```
1  <!-- In runsettings.xml -->
2  <RunSettings>
3    <Playwright>
4      <BrowserName>chromium</BrowserName>
5      <StorageStatePath>playwright/.auth/user.json</StorageStatePath>
6    </Playwright>
7  </RunSettings>
```

```
1  // In Test-Konfiguration
2  [TestClass]
3  public class AuthenticatedTests : PageTest
4  {
5      public override BrowserNewContextOptions ContextOptions()
6      {
7          return new BrowserNewContextOptions
8          {
9              StorageStatePath = "playwright/.auth/user.json"
10          };
11      }
12  }
```

Mehrere Benutzerrollen in Projekten

```
1  // Verschiedene Testklassen für verschiedene Rollen
2  [TestClass]
3  public class AdminTests : PageTest
4  {
5      public override BrowserNewContextOptions ContextOptions()
6      {
7          return new BrowserNewContextOptions
8          {
9              StorageStatePath = "auth-admin.json"
10         };
11     }
12 }
13
14 [TestClass]
15 public class EditorTests : PageTest
16 {
17     public override BrowserNewContextOptions ContextOptions()
18     {
19         return new BrowserNewContextOptions
20         {
21             StorageStatePath = "auth-editor.json"
```


Browser Context: Isolierte Testumgebungen für Auth & Parallele Tests

- **Browser Context:** Ermöglicht isolierte Browser-Sitzungen
 - Jeder Context hat eigene Cookies, Storage & Auth-Status
 - Perfekt für parallele Tests mit verschiedenen Benutzerrollen
 - Verhindert Interferenzen zwischen Testfällen
- **Auth-Vorteile:**
 - Separate Login-Sessions pro Context
 - Storage State kann pro Context gespeichert werden
 - Einfaches Testen von Multi-User-Szenarien
- **Best Practices:**
 - Context nach Test beenden mit `context.CloseAsync()`
 - Storage State für Auth-Status wiederverwenden

Verschiedene Benutzerrollen verwalten

```
1 // Tests/MultiRoleTests.cs
2 using Microsoft.Playwright;
3 using Microsoft.Playwright.MSTest;
4
5 [TestClass]
6 public class MultiRoleTests : BrowserTest
7 {
8     [TestMethod]
9     public async Task AuthenticateMultipleRoles()
10    {
11        // Admin-Kontext mit gespeicherter Authentifizierung
12        var adminContext = await Browser.NewContextAsync(new()
13        {
14            StorageStatePath = "playwright/.auth/admin.json"
15        });
16        var adminPage = await adminContext.NewPageAsync();
17
18        // User-Kontext mit gespeicherter Authentifizierung
19        var userContext = await Browser.NewContextAsync(new()
20        {
21            StorageStatePath = "playwright/.auth/user.json"
```

Übung 3 – Login automatisieren mit Storage State

Ziel: Effizienten Login durch Speichern/Wiederverwenden des Auth-Status.

Aufgaben:

1. Authentifizierungs-Test erstellen, der UI-Login durchführt und `Context.StorageStateAsync()` verwendet.
2. `ContextOptions()` Method überschreiben, um `StorageStatePath` zu nutzen.
3. Authentifizierten Test schreiben, der direkt auf geschützte Seite (`/news/private`) zugreift.

Zeit: 30 Minuten

Übung 3B – API-basierte Authentifizierung

Ziel: Login-Setup durch API-Authentifizierung optimieren.

Aufgaben:

1. API-Login über `Playwright.APIRequest.NewContextAsync()` und `PostAsync()` durchführen.
2. Credentials aus Umgebungsvariablen verwenden (`Environment.GetEnvironmentVariable()`).
3. `request.StorageStateAsync()` verwenden, um Auth-Status zu speichern.
4. Tests schreiben, die gespeicherten Auth-Status für UI und API Tests nutzen.

Zeit: 30 Minuten

Fixtures – Einführung & Nutzen

- **Fixtures:** Wiederverwendbare Testumgebungen
- **Vorteile:**
 - Setup und Teardown Automatisierung
 - Ressourcen-Sharing zwischen Tests
 - Testcode-Vereinfachung
 - Bessere Testorganisation

TestMethod

```
1  public async Task MitEingebautenFixtures()  
2  {  
3      // Page, Context, Browser sind als Properties verfügbar  
4      await Page.GotoAsync("https://example.com");  
5      // ...  
6  }
```

Eingebaute Playwright-Fixtures

Fixture	Beschreibung	Scope
<code>page</code>	Einzelne Browserseite	Test
<code>context</code>	Browser-Kontext (wie Inkognito)	Test
<code>browser</code>	Browser-Instanz	Worker
<code>browserName</code>	Name des Browsers (chromium, firefox, webkit)	Worker
<code>playwright</code>	Playwright-API-Instanz	Worker
<code>request</code>	API für HTTP-Anfragen	Worker

Eigene Fixtures erstellen

```
1  // BaseTestFixture.cs
2  using Microsoft.Playwright;
3  using Microsoft.Playwright.MSTest;
4
5  public class BaseTestFixture : PageTest
6  {
7      protected TestUser TestUser { get; private set; } = new TestUser
8      {
9          Username = "testuser",
10         Email = "test@example.com"
11     };
12
13     // Fixture für eingeloggten User
14     protected async Task<IPage> GetLoggedInPageAsync()
15     {
16         // Setup: Login durchführen
17         await Page.GotoAsync("/login");
18         await Page.GetByLabel("Email").FillAsync("test@example.com");
19         await Page.GetByLabel("Password").FillAsync("password");
20         await Page.ClickAsync("#login-button");
21     }
```

Fixture-Optionen definieren

```
1  // BaseTestFixture.cs
2  using Microsoft.Playwright;
3  using Microsoft.Playwright.MSTest;
4
5  public abstract class BaseTestFixture : PageTest
6  {
7      // Fixture-Optionen als Properties definieren
8      protected virtual UserRole UserRole { get; } = UserRole.Viewer;
9      protected virtual bool DatabaseReset { get; } = true;
10
11     // Setup/Teardown basierend auf Optionen
12     [TestInitialize]
13     public virtual async Task SetupWithOptions()
14     {
15         if (DatabaseReset)
16         {
17             // Database reset logic
18         }
19     }
20 }
```


Rollenbasierte Authentifizierungs-Fixture

```
1 // RoleBasedTestFixture.cs
2 public class RoleBasedTestFixture : BaseTestFixture
3 {
4     // Fixture, die eine Option verwendet
5     protected async Task<IPage> GetAuthenticatedPageAsync()
6     {
7         // Login basierend auf Rolle
8         await Page.GotoAsync("/login");
9
10        switch (UserRole)
11        {
12            case UserRole.Admin:
13                await Page.GetByLabel("Email").FillAsync("admin@example.com");
14                await Page.GetByLabel("Password").FillAsync("adminpass");
15                break;
16            case UserRole.Editor:
17                await Page.GetByLabel("Email").FillAsync("editor@example.com");
18                await Page.GetByLabel("Password").FillAsync("editorpass");
19                break;
20            case UserRole.Viewer:
21                await Page.GetByLabel("Email").FillAsync("viewer@example.com");
```

Testdaten-Fixture mit Optionen

```
1  // TestDataFixture.cs (Fortsetzung)
2  public class TestDataFixture : BaseTestFixture
3  {
4      // Fixture, die eine Option verwendet
5      protected async Task<TestData> PrepareTestDataAsync()
6      {
7          // Testdaten vorbereiten
8          TestData data;
9
10         if (DatabaseReset)
11         {
12             // Datenbank zurücksetzen und Testdaten laden
13             data = await SetupFreshTestDataAsync();
14         }
15         else
16         {
17             // Existierende Daten verwenden
18             data = await GetExistingTestDataAsync();
19         }
20
21         return data;
```

Fixture-Abhängigkeiten und Komposition

```
1  // PageObjectTestFixture.cs
2  using Microsoft.Playwright;
3  using Microsoft.Playwright.MSTest;
4
5  public class PageObjectTestFixture : PageTest
6  {
7      // Page Objects als Properties
8      protected LoginPage LoginPage { get; private set; } = null!;
9      protected DashboardPage DashboardPage { get; private set; } = null!;
10
11     // User-Fixtures
12     protected TestUser AuthenticatedUser { get; private set; } = null!;
13     protected TestUser AdminUser { get; private set; } = null!;
14
15     [TestInitialize]
16     public async Task SetupPageObjects()
17     {
18         // Page-Object-Fixtures
19         LoginPage = new LoginPage(Page);
20         DashboardPage = new DashboardPage(Page);
21     }
```

Beispiel: Fixture für Login-Setup

```
1  // Tests/ExampleTests.cs
2  using Microsoft.Playwright.MSTest;
3
4  [TestClass]
5  public class ExampleTests : PageObjectTestFixture
6  {
7      [TestMethod]
8      public async Task DashboardZeigtBenutzerinfoAn()
9      {
10         var loggedInPage = await GetLoggedInPageAsync();
11         await loggedInPage.GotoAsync("/dashboard");
12
13         var usernameElement = loggedInPage.GetByTestId("username");
14         await Expect(usernameElement).ToBeVisibleAsync();
15         await Expect(usernameElement).ToHaveTextAsync(AuthenticatedUser.Name);
16     }
17
18     [TestMethod]
19     public async Task ProfilKannBearbeitetWerden()
20     {
21         var loggedInPage = await GetLoggedInPageAsync();
```

Übung 3C – Eigene Fixtures erstellen

Ziel: Wiederkehrende Setup-Schritte mit eigenen Fixtures kapseln.

Aufgaben:

1. Fixture-Klasse (`FeedTestFixture.cs`) erstellen und von `PageTest` erben.
2. Fixtures definieren (z.B. `PublicFeeds` für vorbereitete Public News Seite, `FeedApi` für authentifizierten API-Client).
3. Tests schreiben, die diese Fixtures verwenden.

Zeit: 40 Minuten

Test-Strukturierung vor dem POM

Warum strukturierten Testcode schreiben?

- **Probleme unstrukturierter Tests:**
 - Duplizierter Code für wiederkehrende Aktionen und Locators
 - Schwer zu lesen, zu verstehen und zu warten
 - Änderungen an der UI erfordern Anpassungen an vielen Stellen
 - Höhere Fehleranfälligkeit mit zunehmender Testanzahl
- **Evolutionsstufen der Teststrukturierung:**
 1. **Einfache Tests:** Direkter Code ohne besondere Struktur
 2. **Hilfsfunktionen:** Wiederverwendbare Funktionen in der Testdatei
 3. **Fixture-basiert:** Wiederkehrendes Setup in Fixtures auslagern
 4. **Page Object Model:** Vollständige Kapselung von Seiteninteraktionen

Erste Schritte: Hilfsfunktionen

```
1 // Duplizierter, unstrukturierter Code:
2 await Page.GotoAsync("https://demo.playwright.dev/todomvc/");
3 await Page.GetByPlaceholder("What needs to be done?").FillAsync("Todo 1");
4 await Page.Keyboard.PressAsync("Enter");
5
6 // Besser: Hilfsfunktion zur Wiederverwendung
7 public async Task CreateTodoAsync(IPage page, string text)
8 {
9     await page.GetByPlaceholder("What needs to be done?").FillAsync(text);
10    await page.Keyboard.PressAsync("Enter");
11 }
12
13 // Verwendet in Tests:
14 [TestMethod]
15 public async Task TodoAnlegen()
16 {
17     await Page.GotoAsync("https://demo.playwright.dev/todomvc/");
18     await CreateTodoAsync(Page, "Todo 1");
19     await Expect(Page.GetByText("Todo 1")).ToBeVisibleAsync();
20 }
```


Vor- und Nachteile einfacher Hilfsfunktionen

Vorteile

- ✓ Einfach zu implementieren
- ✓ Reduziert Code-Duplikation
- ✓ Bessere Lesbarkeit
- ✓ Einfacher Einstieg in Strukturierung
- ✓ Tests bleiben übersichtlich

Nachteile

- ✗ Funktionen nur innerhalb einer Datei nutzbar
- ✗ Keine klare Kapselung von Zuständen
- ✗ Bei komplexen UIs schnell unübersichtlich
- ✗ Keine Standardisierung über Testdateien hinweg
- ✗ Nicht ideal für größere Projekte

Tipp

Hilfsfunktionen sind ein guter erster Schritt zur besseren Testorganisation. Bei wachsender Komplexität ist jedoch der Übergang zum Page Object Model sinnvoll!

Übung X – Code-Chaos bändigen

Ziel: Verbessere die Teststruktur mit Hilfsfunktionen.

Demo-Website: <https://demo.playwright.dev/todomvc/>

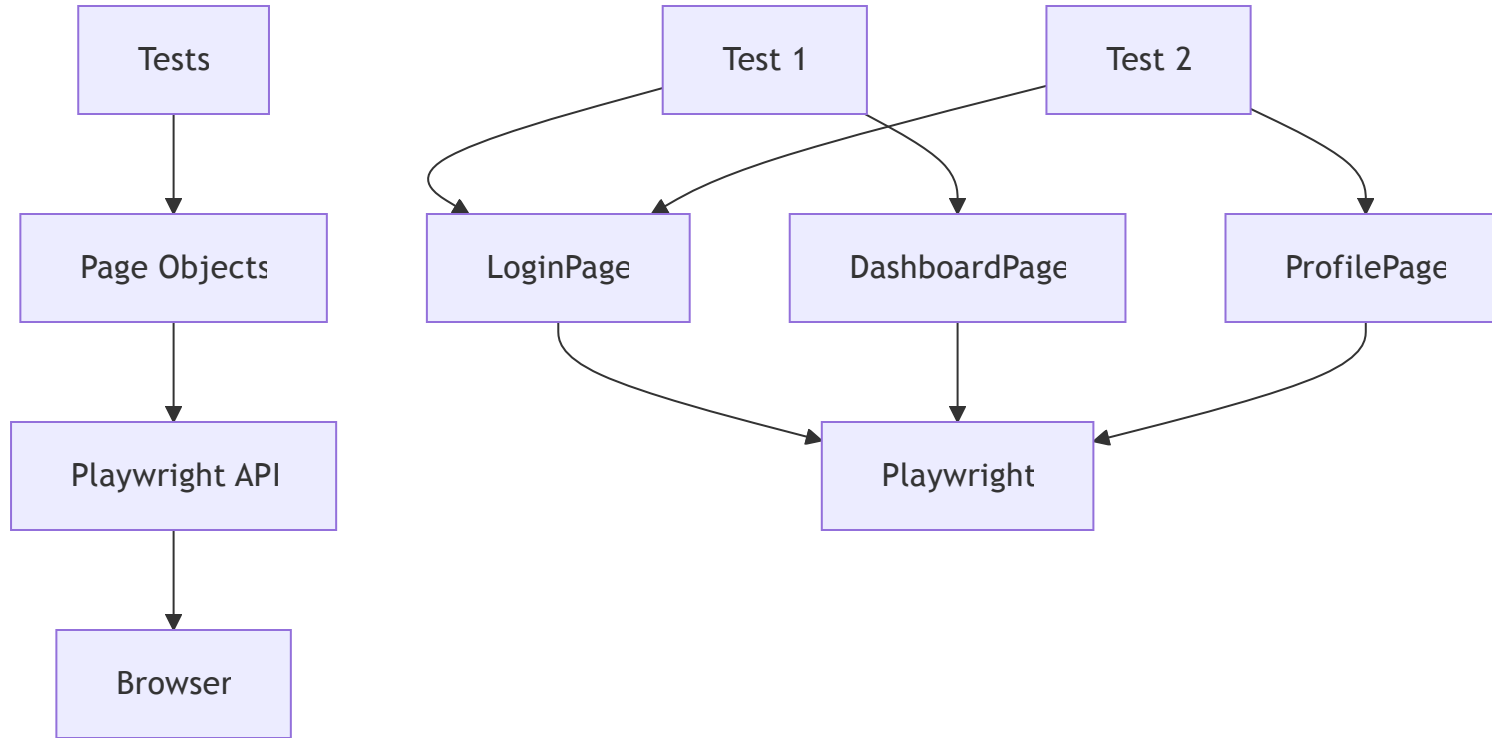
Aufgaben:

1. Erstelle eine neue Testdatei `Tests/ToDoHelperTests.cs` mit wiederholenden Testschritten:
 - Ein Test soll mehrere Todos anlegen und filtern
 - Ein Test soll Todos anlegen, bearbeiten und löschen
 - Beides ohne Hilfsfunktionen (mit viel Code-Wiederholung)
2. Identifiziere wiederkehrende Aktionen und Locators:
 - Extrahiere sie in Hilfsmethoden innerhalb der Testklasse
 - Beispiel: `CreateTodoAsync()` , `ToggleTodoAsync()` , `DeleteTodoAsync()`
 - Beispiel: Klasse für gemeinsame Locators
3. Refaktoriere deine Tests, um diese Hilfsmethoden zu verwenden
 - Vergleiche Lesbarkeit und Wartbarkeit vor und nach dem Refactoring

Page Object Model – Einführung

- **Design Pattern** zur Strukturierung von Tests
- **Abstrahiert** Seiteninteraktion in Klassen
- **Kapselt** Seitenelemente und -funktionen
- **Vermeidet** Code-Duplikation
- **Vereinfacht** Test-Wartung

Page Object Model – Architektur



- **Tests:** Beschreiben Geschäftslogik und Erwartungen

Nachbesprechung: POM Intro

Diskussion der Übungsergebnisse

Alternativen zum Page Object Model

Component Object Model

- Fokus auf UI-Komponenten statt Seiten
- Besser für SPAs und komplexe UIs
- Wiederverwendbare Komponenten
- Beispiel: Header, Sidebar, Modal

App Actions Pattern

- Fokus auf Benutzeraktionen
- Abstrahiert UI-Details
- API-ähnliche Schnittstelle
- Beispiel: `app.login()` , `app.createTodo()`

```
1  public class TodoItem
2  {
3      private readonly ILocator _root;
4
5      public TodoItem(ILocator root)
6      {
7          _root = root;
8      }
9
10     public async Task ToggleAsync()
11     {
12         await _root.GetByRole(AriaRole.Checkbox).ClickAsync();
13     }
14
15     public async Task DeleteAsync()
16     {
17         await _root.GetByRole(AriaRole.Button).ClickAsync();
18     }
19
20     public async Task<string?> GetTextAsync()
21     {
```

Vorteile des Page Object Models

- **Wartbarkeit:** Änderungen nur an einer Stelle
- **Lesbarkeit:** Tests beschreiben "Was", nicht "Wie"
- **Wiederverwendbarkeit:** Funktionen in mehreren Tests nutzen
- **Stabilität:** Reduziert Fehleranfälligkeit
- **Organisation:** Strukturierte Testarchitektur

Live-Demo: POM Refactoring

Praktische Demonstration der Konzepte

Best Practices für Page Objects

- **Verantwortlichkeiten trennen:**
 - Page Objects: UI-Interaktionen
 - Tests: Assertions und Geschäftslogik
- **Methoden sinnvoll benennen:**
 - Geschäftsprozesse abbilden & Domänensprache verwenden
 - Beispiel: `loginAsAdmin()` statt `fillEmailAndPassword()`
- **Rückgabewerte nutzen:**
 - Methoden geben neue Page Objects zurück
 - Beispiel: `loginPage.login().shouldShowDashboard()`
- **Assertions vermeiden:**
 - Page Objects sollten keine Assertions enthalten
 - Ausnahme: Hilfsmethoden wie `shouldBeVisible()`

Page Object Model – Beispielcode

```
1  using Microsoft.Playwright;
2
3  public class LoginPage
4  {
5      private readonly IPage _page;
6      private readonly ILocator _emailInput;
7      private readonly ILocator _passwordInput;
8      private readonly ILocator _loginButton;
9
10     public LoginPage(IPage page)
11     {
12         _page = page;
13         _emailInput = page.GetByLabel("Email");
14         _passwordInput = page.GetByLabel("Passwort");
15         _loginButton = page.GetByRole(AriaRole.Button, new() { Name = "Anmelden" });
16     }
17
18     public async Task GotoAsync()
19     {
20         await _page.GotoAsync("/login");
21     }
22 }
```

Umsetzung im Projekt

```
1  // Tests/LoginTests.cs
2  using Microsoft.Playwright;
3  using Microsoft.Playwright.MSTest;
4  using System.Text.RegularExpressions;
5
6  [TestClass]
7  public class LoginTests : PageTest
8  {
9      [TestMethod]
10     public async Task ErfolgreichenLogin()
11     {
12         var loginPage = new LoginPage(Page);
13         var dashboardPage = new DashboardPage(Page);
14
15         await loginPage.GotoAsync();
16         await loginPage.LoginAsync("user@example.com", "password123");
17
18         // Prüfen, ob wir zum Dashboard weitergeleitet wurden
19         await Expect(Page).ToHaveURLAsync(new Regex("dashboard"));
20         await Expect(dashboardPage.WelcomeMessage).ToBeVisibleAsync();
21     }
22 }
```

Übung 4 – Refactoring mit Page Objects

Ziel: Bestehende Tests (Übung 2) mit Page Object Model refaktorisieren.

Aufgaben:

1. `PublicNewsPage` Klasse erstellen (`Tests/Pages/PublicNewsPage.cs`).
2. Locators und Interaktionsmethoden (Suche, Filter) kapseln.
3. Hilfsmethoden für Datenextraktion (Anzahl, Titel, Kategorie) hinzufügen.
4. Tests aus Übung 2 refaktorisieren, um `PublicNewsPage` zu verwenden.

Zeit: 30 Minuten

Nachbesprechung: POM Best Practices

Diskussion der Übungsergebnisse

Zusammenfassung Tag 2 & Ausblick Tag 3

- Debugging-Techniken
- Authentifizierung & Storage State
- Fixtures für Setup/Teardown
- Page Object Model Grundlagen
- **Morgen:** Fortgeschrittenes POM, API Mocking, Testing-Strategien, Mobile, CI/CD

Fortgeschrittene Themen & Testing-Strategien

Tag 3: Advanced POM, API Mocking, Datenmanagement, Mobile, CI/CD



Wiederholung: Tag 2 Konzepte

- **Debugging:** Inspector, Trace Viewer, `Page.PauseAsync()`
- **Authentifizierung:** UI vs. API, Storage State, Global Setup
- **Fixtures:** Eingebaute (`page` , `context`), Eigene (`base.extend`), Scopes
- **Page Object Model:** Grundlagen, Architektur, Best Practices
- **Offene Fragen aus Tag 2?**

Tagesagenda im Detail

Vormittag

- Fortgeschrittene POM Patterns (Komponenten, Fluent Interface)
- API Mocking (`page.route`)
- Testing-Strategien (Datenmanagement, Reset)
- Praktische Übungen (Advanced POM, API Mocking)

Nachmittag

- Mobile Testing (Emulation, Responsive, Touch)
- Parameterisierte Tests
- Kurzer Ausblick: CI/CD
- Praktische Übungen (Mobile, Parameterisierte Tests)
- Zusammenfassung & Abschluss

Fortgeschrittene Page Object Patterns

```
1  // Basis Page Object
2  using Microsoft.Playwright;
3
4  public abstract class BasePage
5  {
6      protected readonly IPage Page;
7
8      protected BasePage(IPage page)
9      {
10         Page = page;
11     }
12
13     public async Task WaitForPageLoadAsync()
14     {
15         await Page.WaitForLoadStateAsync(LoadState.NetworkIdle);
16     }
17
18     // Navigation mit Typensicherheit
19     public async Task<T> NavigateToAsync<T>(string url) where T : BasePage
20     {
21         await Page.GotoAsync(url);
```

Erweitertes Page Object Beispiel

```
1  // Pages/ToDoPage.cs
2  using Microsoft.Playwright;
3
4  public class ToDoPage
5  {
6      private readonly IPage _page;
7      private readonly ILocator _newTodoInput;
8      private readonly ILocator _todoItems;
9      private readonly Dictionary<string, ILocator> _filterButtons;
10
11     public ToDoPage(IPage page)
12     {
13         _page = page;
14         _newTodoInput = page.GetByPlaceholder("What needs to be done?");
15         _todoItems = page.Locator(".todo-list li");
16         _filterButtons = new Dictionary<string, ILocator>
17         {
18             ["active"] = page.GetByRole(AriaRole.Link, new() { Name = "Active" }),
19             ["completed"] = page.GetByRole(AriaRole.Link, new() { Name = "Completed" }),
20             ["all"] = page.GetByRole(AriaRole.Link, new() { Name = "All" })
21         };
22     }
23 }
```

Komponenten-basiertes Page Object Modell

```
1  // Components/ToDoItem.cs
2  using Microsoft.Playwright;
3
4  public class ToDoItem
5  {
6      private readonly ILocator _checkbox;
7      private readonly ILocator _label;
8      private readonly ILocator _root;
9
10     public ToDoItem(ILocator root)
11     {
12         _root = root;
13         _checkbox = root.GetByRole(AriaRole.Checkbox);
14         _label = root.Locator("label");
15     }
16
17     public async Task ToggleAsync()
18     {
19         await _checkbox.ClickAsync();
20     }
21 }
```

Fluent Interface Pattern

```
1  using Microsoft.Playwright;
2  using Microsoft.Playwright.MSTest;
3
4  public class LoginPage
5  {
6      private readonly IPage _page;
7
8      public LoginPage(IPage page)
9      {
10         _page = page;
11     }
12
13     public async Task<LoginPage> GotoAsync()
14     {
15         await _page.GotoAsync("/login");
16         return this; // Rückgabe der Instanz für Verkettung
17     }
18
19     public async Task<LoginPage> FillEmailAsync(string email)
20     {
21         await _page.GetByLabel("Email").FillAsync(email);
```

Übung 4B – Erweiterte Page Objects (Komponenten & Fluent Interface)

Ziel: POM verbessern durch Komponenten-Objekte & Fluent Interface.

Aufgaben:

1. `NewsItemComponent` erstellen, das ein einzelnes News-Item kapselt.
2. `PublicNewsPage` anpassen, um `NewsItemComponent` zu verwenden.
3. Fluent Interface (`return this`) in `PublicNewsPage` implementieren.
4. Tests refaktorisieren, um Komponenten und Fluent Interface zu nutzen.

Zeit: 30 Minuten

Nachbesprechung: Advanced POM

Diskussion der Übungsergebnisse

Warum API Mocking?

- **Kontrolle über Testumgebung**
- **Abkopplung** von externen Diensten
- **Testdaten** gezielt manipulieren
- **Edge Cases** und Fehlerzustände simulieren
- **Schnellere Tests** ohne Netzwerkverzögerungen
- **Stabilere Tests** unabhängig von externen Diensten

Arten von Netzwerk-Mocking

Request Mocking

- Anfragen abfangen und modifizieren
- Anfragen blockieren
- Anfragen verzögern
- Anfragen umleiten

Response Mocking

- Antworten simulieren
- Statuscodes ändern
- Antwortdaten manipulieren
- Fehler simulieren

Beispiel: Request & Response Mocking

```
1  // Request Mocking
2  await Page.RouteAsync("**/api/data", async route =>
3  {
4      // Anfrage abfangen und modifizieren
5      var url = route.Request.Url;
6      var newUrl = url.Replace("api/data", "api/mock-data");
7      await route.ContinueAsync(new() { Url = newUrl });
8  });
9
10 // Response Mocking
11 await Page.RouteAsync("**/api/users", async route =>
12 {
13     // Antwort simulieren
14     await route.FulfillAsync(new()
15     {
16         Status = 200,
17         ContentType = "application/json",
18         Body = JsonSerializer.Serialize(new[] { new { id = 1, name = "Test User" } })
19     });
20 });
```

Wann sollte man API Mocking einsetzen?

- **Für Tests mit externen Abhängigkeiten:**
 - Drittanbieter-APIs
 - Zahlungs-Gateways
 - Authentifizierungsdienste
- **Für Tests mit nicht-deterministischen Daten:**
 - Zeitabhängige Daten
 - Zufällige Daten
 - Daten von anderen Benutzern

Route-Matching-Strategien: Exakte URL

```
1 // Exakte URL
2 await Page.RouteAsync("https://example.com/api/users", async route => {
3     // Nur exakte URL wird abgefangen
4 });
```

Route-Matching-Strategien: Wildcards

```
1 // URL-Muster mit Wildcards
2 await Page.RouteAsync("**/api/users", async route => {
3     // Alle URLs, die mit /api/users enden
4 });
```

Route-Matching-Strategien: Regex

```
1 // Regex-Muster
2 await Page.RouteAsync(new Regex(@"api/users/\d+"), async route => {
3     // URLs wie /api/users/123, /api/users/456, etc.
4 });
```

Route-Matching-Strategien: HTTP-Methode

```
1 // Nach HTTP-Methode filtern
2 await Page.RouteAsync("**/api/users", async route => {
3     if (route.Request.Method == "POST") {
4         // Nur POST-Anfragen abfangen
5     } else {
6         // Andere Anfragen normal durchlassen
7         await route.ContinueAsync();
8     }
9 });
```


Route-Matching-Strategien: Ressourcentyp

```
1 // Nach Ressourcentyp filtern
2 await Page.RouteAsync("**/*", async route => {
3     if (route.Request.ResourceType == "image") {
4         // Alle Bildanfragen abfangen
5         await route.AbortAsync();
6     } else {
7         await route.ContinueAsync();
8     }
9 });
```

Page.RouteAsync() – Fehlerfall simulieren

TestMethod

```
1  public async Task ZeigtFehlermeldungBeiApiFehler()
2  {
3      // API-Fehler simulieren
4      await Page.RouteAsync("**/api/data", async route =>
5          await route.FulfillAsync(new()
6              {
7                  Status = 500,
8                  ContentType = "application/json",
9                  Body = JsonSerializer.Serialize(new { error = "Server Error" })
10             });
11
12      await Page.GotoAsync("/dashboard");
13
14      // Prüfen, ob Fehlermeldung angezeigt wird
15      await Expect(Page.GetByTestId("error-message")).ToBeVisibleAsync();
16      await Expect(Page.GetByTestId("error-message")).ToContainTextAsync("Server Error");
17  }
```

Page.RouteAsync() – Ladezustand testen

TestMethod

```
1 public async Task ZeigtLadezustandWaehrendApiAnfrage()  
2 {  
3     // Verzögerte API-Antwort  
4     await Page.RouteAsync("**/api/slowdata", async route => {  
5         await Task.Delay(1000);  
6         await route.FulfillAsync(new() { Status = 200, Body = "{\"data\":\"success\"}" });  
7     });  
8  
9     await Page.GotoAsync("/slowpage");  
10  
11     // Prüfen, ob Ladeindikator angezeigt wird  
12     await Expect(Page.GetByTestId("loading")).ToBeVisibleAsync();  
13 }
```

Übung 5 – API Mocking

Ziel: Netzwerkanfragen mocken für unabhängige Tests & Szenarien-Simulation.

Aufgaben:

1. `Page.RouteAsync("**/api/news/public", ...)` verwenden, um API-Antworten zu mocken.
2. Erfolgsfall mit Mock-Daten simulieren und prüfen.
3. Fehlerfall (Status 500) simulieren und Fehler-UI prüfen.
4. Leere Daten (`items: []`) simulieren und UI prüfen.
5. (Optional) Langsame Antwort simulieren und Ladezustand prüfen.

Zeit: 30 Minuten

Nachbesprechung: API Mocking

Diskussion der Übungsergebnisse

Netzwerkanfragen mit `waitForResponse` abfangen

- **Problem:** Wie können wir auf bestimmte Netzwerkanfragen warten und sie auswerten?
- **Lösung:** `Page.WaitForResponseAsync()` - Wartet auf eine Netzwerkantwort, die einem Muster entspricht
- **Anwendungsfälle:**
 - API-Antworten validieren
 - Daten aus Antworten extrahieren
 - Asynchrone Operationen synchronisieren

Grundlegende Verwendung von waitForResponse

```
1 // Auf eine bestimmte Anfrage warten
2 [TestMethod]
3 public async Task ApiAntwortAbfangen()
4 {
5     // Navigation und Anfrage gleichzeitig ausführen
6     var responseTask = Page.WaitForResponseAsync("**/api/news/**");
7     await Page.GotoAsync("/news");
8
9     // Auf die Antwort warten
10    var response = await responseTask;
11
12    // Antwort validieren
13    Assert.AreEqual(200, response.Status);
14    var data = await response.JsonAsync<dynamic>();
15    Assert.IsTrue(data.items.Count > 0);
16 }
```

Auf Netzwerkanfragen während Interaktionen warten

TestMethod

```
1 public async Task AufApiAntwortNachBenutzeraktionWarten()
2 {
3     await Page.GotoAsync("/dashboard");
4
5     // Auf Antwort warten, während Button geklickt wird
6     var responseTask = Page.WaitForResponseAsync("**/api/refresh-data");
7     await Page.GetByRole(AriaRole.Button, new() { Name = "Daten aktualisieren" }).ClickAsync();
8
9     // Antwort abwarten und prüfen
10    var response = await responseTask;
11    Assert.IsTrue(response.Ok);
12
13    // UI-Updates prüfen, die von der API-Antwort abhängen
14    var data = await response.JsonAsync<dynamic>();
15    await Expect(Page.GetText($"Letzte Aktualisierung: {data.timestamp}")).ToBeVisibleAsync();
16 }
```


waitForResponse mit Prädikaten für präzise Filterung

TestMethod

```
1  public async Task AufSpezifischeApiAntwortenWarten()
2  {
3      await Page.GotoAsync("/products");
4
5      // Auf POST-Anfrage mit bestimmtem Body warten
6      var responseTask = Page.WaitForResponseAsync(response => {
7          return response.Url.Contains("/api/filter") &&
8              response.Request.Method == "POST";
9      });
10
11     // Filter anwenden
12     await Page.GetByLabel("Kategorie").SelectOptionAsync("elektronik");
13     await Page.GetByRole(AriaRole.Button, new() { Name = "Filtern" }).ClickAsync();
14
15     // Antwort verarbeiten
16     var response = await responseTask;
17     var data = await response.JsonAsync<dynamic>();
18
19     // Daten validieren
```

waitForResponse kombiniert mit Mocking

TestMethod

```
1  public async Task ApiMockingMitResponseValidierung()
2  {
3      // API-Antwort mocken
4      await Page.RouteAsync("**/api/search", async route => {
5          await route.FulfillAsync(new()
6          {
7              Status = 200,
8              ContentType = "application/json",
9              Body = JsonSerializer.Serialize(new {
10                  results = new[] {
11                      new { id = 1, name = "Gemocktes Produkt" }
12                  }
13              })
14          });
15      });
16
17      // Auf die gemockte Antwort warten
18      var responseTask = Page.WaitForResponseAsync("**/api/search");
19      await Page.GotoAsync("/search?q=test");
20      var response = await responseTask;
```

Playwright Request API für direktes API Testing

- Playwright bietet nicht nur Browser-Automatisierung, sondern auch direkte **API-Test-Funktionalitäten**
- Die `request` Fixture ermöglicht HTTP-Anfragen ohne Browser-Kontext
- Ideal für:
 - Reine API-Tests
 - API-Prüfungen vor/nach UI-Tests
 - Testdaten-Setup via API
 - Backend-Validierung
- Unterstützt alle HTTP-Methoden: GET, POST, PUT, DELETE, etc.

Grundlegende API-Anfragen mit Request

```
1  using Microsoft.Playwright.MSTest;
2  using Microsoft.VisualStudio.TestTools.UnitTesting;
3
4  [TestMethod]
5  public async Task BasicApiTesting()
6  {
7      // GET-Anfrage
8      var getResponse = await Request.GetAsync("https://jsonplaceholder.typicode.com/posts/1");
9
10     // Validierung
11     Assert.IsTrue(getResponse.Ok);
12     Assert.AreEqual(200, getResponse.Status);
13
14     var responseBody = await getResponse.JsonAsync<dynamic>();
15     Assert.AreEqual(1, responseBody.id);
16     Assert.IsNotNull(responseBody.title);
17 }
```

- **request.get():** Führt eine GET-Anfrage durch
- **response.ok():** Prüft, ob Status im Bereich 200-299 liegt

Request-Methoden mit JSONPlaceholder

TestMethod

```
1  public async Task CrudOperationsWithApi()
2  {
3      // Create: POST-Anfrage
4      var createResponse = await Request.PostAsync("https://jsonplaceholder.typicode.com/posts", new()
5      {
6          DataObject = new {
7              title = "foo",
8              body = "bar",
9              userId = 1
10         }
11     });
12     Assert.IsTrue(createResponse.Ok);
13     var newPost = await createResponse.JsonAsync<dynamic>();
14     Assert.IsNotNull(newPost.id);
15
16     // Read: GET-Anfrage
17     var getResponse = await Request.GetAsync($"https://jsonplaceholder.typicode.com/posts/{newPost.id}");
18     Assert.IsTrue(getResponse.Ok);
19
20     // Update: PUT-Anfrage
21     var updateResponse = await Request.PutAsync($"https://jsonplaceholder.typicode.com/posts/{newPost.id}", new()
```

Erweiterte Request-Optionen

TestMethod

```
1  public async Task ApiRequestWithAdvancedOptions()
2  {
3      // Anfrage mit verschiedenen Optionen
4      var response = await Request.GetAsync("https://jsonplaceholder.typicode.com/posts", new()
5      {
6          // URL-Parameter
7          Params = new Dictionary<string, object>
8          {
9              ["userId"] = 1,
10             ["_limit"] = 5
11         },
12
13         // Request-Header
14         Headers = new Dictionary<string, string>
15         {
16             ["Accept"] = "application/json",
17             ["Custom-Header"] = "test-value"
18         },
19
20         // Timeout setzen (in Millisekunden)
21         Timeout = 20000
```

Request Fixtures konfigurieren

```
1  <!-- .runsettings or similar .NET configuration -->
2  <configuration>
3    <appSettings>
4      <!-- Basis-URL für alle Anfragen -->
5      <add key="BaseUrl" value="https://jsonplaceholder.typicode.com" />
6      <!-- Extrapolierte Header für alle Anfragen -->
7      <add key="DefaultHeaders:Accept" value="application/json" />
8      <add key="DefaultHeaders:Authorization" value="Bearer test-token" />
9    </appSettings>
10 </configuration>
11
12 // Im Test
13 [TestMethod]
14 public async Task UsingBaseUrlFromConfig()
15 {
16     // Vollständige URL: https://jsonplaceholder.typicode.com/posts
17     var response = await Request.GetAsync("/posts");
18     Assert.IsTrue(response.Ok);
19 }
```

API-Schema-Validierung mit JSONSchema

```
1  using Microsoft.Playwright.MSTest;
2  using Microsoft.VisualStudio.TestTools.UnitTesting;
3  using Newtonsoft.Json.Schema;
4  using Newtonsoft.Json.Linq;
5
6  [TestMethod]
7  public async Task ValidateApiResponseAgainstJsonSchema()
8  {
9      var response = await Request.GetAsync("https://jsonplaceholder.typicode.com/posts/1");
10     Assert.IsTrue(response.Ok);
11
12     var responseBody = await response.TextAsync();
13
14     // JSON Schema für einen Post
15     var postSchema = JSchema.Parse(@"{
16         'type': 'object',
17         'required': ['id', 'title', 'body', 'userId'],
18         'properties': {
19             'id': { 'type': 'integer' },
20             'title': { 'type': 'string' },
21             'body': { 'type': 'string' }
```


Übung: API Testing mit JSONPlaceholder

Ziel: Die Playwright Request API für direkte API-Tests mit JSONPlaceholder nutzen.

Aufgaben:

1. Erstelle einen neuen Testfile `ApiTests.cs` und verwende die `Request` Property.
2. Teste folgende Endpunkte:
 - GET `/todos` - Prüfe Statuscode und ob die Antwort ein Array ist
 - GET `/todos/1` - Prüfe Details des ersten Todos
 - POST `/todos` - Erstelle ein neues Todo und validiere die Antwort
3. Erstelle einen Test, der die Beziehung zwischen Ressourcen testet:
 - Hole einen Benutzer via GET `/users/1`
 - Finde alle Posts dieses Benutzers via GET `/users/1/posts`
 - Prüfe, ob die Posts tatsächlich dem Benutzer gehören
4. **Bonus:** Implementiere einen test mit Schema-Validierung für `/users` und `/posts`.

Zeit: 30 Minuten

Kombiniertes API & UI Testing

- Besonders effektiver Ansatz: API- und UI-Tests **kombinieren**
- Backend via API testen, Frontend über UI testen
- Schnelleres **Testdaten-Setup** durch API-Aufrufe
- **Vorvalidierung**: API-Status prüfen, bevor UI-Tests starten
- **Vollständige Testabdeckung**: Backend UND Frontend

Beispiel: Kombiniertes API & UI Testing

```
1 using Microsoft.Playwright.MSTest;
2 using Microsoft.VisualStudio.TestTools.UnitTesting;
3
4 [TestMethod]
5 public async Task KombiniertesApiUndUiTest()
6 {
7     // 1. API-Anfrage: Daten abrufen
8     var response = await Request.GetAsync("https://jsonplaceholder.typicode.com/posts?_limit=5");
9     Assert.IsTrue(response.Ok);
10    var posts = await response.JsonAsync<dynamic[]>();
11
12    // 2. Überprüfen, ob Daten vorhanden sind
13    Assert.AreEqual(5, posts.Length);
14    var firstPostTitle = posts[0].title;
15
16    // 3. UI-Test: Seite aufrufen und Daten validieren
17    await Page.GotoAsync("https://jsonplaceholder.typicode.com/");
18
19    // 4. Mit der API einen neuen Post erstellen
20    var createResponse = await Request.PostAsync("https://jsonplaceholder.typicode.com/posts", new()
21    {
```

Testdaten über API vorbereiten

```
1  using Microsoft.Playwright.MSTest;
2  using Microsoft.VisualStudio.TestTools.UnitTesting;
3  using System.Text.Json;
4
5  public class PostDetailsTests : PageTest
6  {
7      private dynamic? _testPost;
8
9      // Test Setup mit API-Anfragen
10     [TestInitialize]
11     public async Task SetupTestData()
12     {
13         // Testdaten vorbereiten
14         var createResponse = await Request.PostAsync("https://jsonplaceholder.typicode.com/posts", new()
15         {
16             DataObject = new {
17                 title = "Testpost für UI-Test",
18                 body = "Dieser Post wird für UI-Tests verwendet",
19                 userId = 1
20             }
21         });
```

API-Fixtures für Tests definieren

```
1  using Microsoft.Playwright.MSTest;
2  using Microsoft.VisualStudio.TestTools.UnitTesting;
3
4  public class ApiTestDataFixtures : PageTest
5  {
6      protected dynamic[]? TestPosts { get; private set; }
7      protected dynamic? TestUser { get; private set; }
8
9      [TestInitialize]
10     public async Task SetupTestFixtures()
11     {
12         // Fixture: Array von Testposts
13         var postsResponse = await Request.GetAsync("https://jsonplaceholder.typicode.com/posts?_limit=3");
14         TestPosts = await postsResponse.JsonAsync<dynamic[]>();
15
16         // Fixture: Ein Testbenutzer
17         var userResponse = await Request.GetAsync("https://jsonplaceholder.typicode.com/users/1");
18         TestUser = await userResponse.JsonAsync<dynamic>();
19     }
20 }
```

API Response Validierung: Status Code

- **Warum wichtig?** Sicherstellen, dass Ihre Anwendung korrekt auf verschiedene HTTP-Statuscodes reagiert (z.B. 200 OK, 201 Created, 404 Not Found, 500 Server Error).
- **Beim Mocking:** Sie kontrollieren den Status über `route.FulfillAsync(new() { Status = ... })`. Der Test validiert dann das UI-Verhalten basierend auf dem gemockten Status.
- **Bei direkten API-Tests:** Mit Playwright's `request` Fixture können Sie den Status einer echten oder gemockten API-Antwort explizit prüfen.

API Response Validierung: Status Code

```
1 // Beispiel: UI-Test mit gemocktem Fehlerstatus
2 [TestMethod]
3 public async Task UiReagiertAufServerfehler500()
4 {
5     await Page.RouteAsync("**/api/data", async route =>
6         await route.FulfillAsync(new() { Status = 500, Body = "Internal Server Error" })
7     );
8     await Page.GotoAsync("/data-display-page"); // Annahme einer Seite, die Daten anzeigt
9     await Expect(Page.GetByText("Ein Fehler ist aufgetreten")).ToBeVisibleAsync();
10 }
11
12 // Beispiel: Direkter API-Test mit Playwrights request fixture
13 [TestMethod]
14 public async Task ApiEndpunktLiefertKorrektenStatusCode()
15 {
16     var response = await Request.GetAsync("/api/health-check"); // Ein Beispiel-Endpunkt
17
18     // Überprüft, ob der Statuscode im Bereich 200-299 liegt
19     Assert.IsTrue(response.Ok);
20
21     // Überprüft einen spezifischen Statuscode
```

API Response Validierung: Response Body

- **Warum wichtig?** Überprüfen, ob die API die erwarteten Daten im korrekten Format und Inhalt liefert.
- **Beim Mocking:** Sie definieren den Body mit `route.FulfillAsync(new() { Body = JsonSerializer.Serialize(...), ContentType = "application/json" })`. Der Test validiert, wie die UI diese Daten darstellt.
- **Bei direkten API-Tests:** Die `request` Fixture ermöglicht die detaillierte Überprüfung des Antwort-Bodys (z.B. JSON-Objekte, Textinhalte).

API Response Validierung: Response Body

```
1 // Beispiel: UI-Test mit gemockten JSON-Daten
2 [TestMethod]
3 public async Task UiZeigtGemockteProduktdatenKorrektAn()
4 {
5     var mockProduct = new { id = "prod123", name = "Super Gadget", price = 29.99 };
6     await Page.RouteAsync("**/api/products/prod123", async route =>
7     {
8         await route.FulfillAsync(new()
9         {
10             Status = 200,
11             ContentType = "application/json",
12             Body = JsonSerializer.Serialize(mockProduct)
13         });
14     });
15     await Page.GotoAsync("/products/prod123"); // Annahme einer Produkt-Detailseite
16
17     await Expect(Page.GetByRole(AriaRole.Heading, new() { Name = "Super Gadget" })).ToBeVisibleAsync();
18     await Expect(Page.GetByText($"Preis: {mockProduct.price} €")).ToBeVisibleAsync();
19 }
20 // Beispiel: Direkter API-Test zum Prüfen des Response Body
21 [TestMethod]
```

Testing-Strategien: Datenmanagement

Warum ist Testdaten-Management wichtig?

- **Konsistenz:** Jeder Testlauf sollte unter gleichen Bedingungen starten.
- **Isolation:** Tests sollten sich nicht gegenseitig beeinflussen.
- **Wiederholbarkeit:** Tests müssen zuverlässig reproduzierbar sein.
- **Performance:** Setup und Teardown sollten effizient sein.
- **Realismus:** Testdaten sollten realen Szenarien ähneln.

Strategien zur Testdaten-Generierung

1. Statisches Seeding

- Feste Daten in DB laden (vor Testsuite)
- Einfach, aber unflexibel
- Gefahr von Abhängigkeiten

2. Data Factories

- Code zum Erstellen von Daten (z.B. mit Faker.js)
- Flexibel, anpassbar
- Erfordert Wartung

3. API-basiertes Setup

- Daten über App-API erstellen
- Realistisch, testet API mit
- Langsamer als DB-Zugriff

Datenbank-Reset-Strategien

Vor/Nach jedem Test

- `[TestInitialize]` / `[TestCleanup]` : DB leeren/neu befüllen
- **Vorteil:** Maximale Isolation
- **Nachteil:** Sehr langsam bei vielen Tests

Datenbank-Transaktionen

- `[TestInitialize]` : Transaktion starten
- `[TestCleanup]` : Rollback durchführen
- **Vorteil:** Schnell, gute Isolation
- **Nachteil:** Nicht alle DBs/Setups unterstützen das gut

Vor/Nach der Testsuite

- `[ClassInitialize]` / `[ClassCleanup]` : Einmaliges Setup/Cleanup
- **Vorteil:** Schnell
- **Nachteil:** Keine Isolation zwischen Tests, Reihenfolge wichtig

Testdaten-Management mit Fixtures

- **Ideal für Setup/Teardown:** Kapselt Logik zur Datenerstellung und -bereinigung.
- **Wiederverwendbar:** Gleiche Daten-Setups für mehrere Tests.
- **Konfigurierbar:** Fixture-Optionen für Variationen (z.B. User-Rollen).
- **Isolation:** Test-Scoped Fixtures sorgen für saubere Daten pro Test.

```
1  // Beispiel: Fixture für Testbenutzer mit API-Setup/Teardown
2  public class TestUserFixture : PageTest
3  {
4      protected dynamic? TestUser { get; private set; }
5
6      [TestInitialize]
7      public async Task CreateTestUser()
8      {
9          // Setup: Benutzer über API erstellen
10         var email = $"test-{DateTimeOffset.Now.ToUnixTimeMilliseconds()}@example.com";
11         var response = await Request.PostAsync("/api/users", new()
12         {
13             DataObject = new { email = email, password = "password123" }
14         });
15         TestUser = await response.JsonAsync<dynamic>();
```

Beispiel: Testdaten-Fixture

```
1  // TestDataFixtures.cs
2  using Microsoft.Playwright.MSTest;
3  using Microsoft.VisualStudio.TestTools.UnitTesting;
4
5  public class TestDataFixtures : TestUserFixture
6  {
7      protected dynamic? TestArticle { get; private set; }
8
9      [TestInitialize]
10     public async Task CreateTestArticle()
11     {
12         await base.CreateTestUser(); // Sicherstellen, dass TestUser verfügbar ist
13
14         // Setup: Artikel für den Testbenutzer erstellen
15         var articleData = new {
16             title = "Test Article Title",
17             content = "Test content ... ",
18             authorId = TestUser?.id // Nutzt andere Fixture!
19         };
20         TestArticle = await ApiHelpers.CreateTestArticleAsync(Request, articleData);
21     }
```

Strategie: Tests mit Tags organisieren

- **Warum Tests taggen?**
 - Bessere Organisation von Testsuiten
 - Gezielte Ausführung nach Testtyp oder Wichtigkeit
 - Zeitersparnis durch Filtern in CI/CD-Pipelines
 - Bessere Dokumentation der Testabdeckung
- **Typische Tag-Kategorien:**
 - Nach **Testtyp**: @api , @ui , @visual , @smoke , @regression
 - Nach **Laufzeit**: @slow , @fast
 - Nach **Bereich**: @auth , @payment , @profile
 - Nach **Priorität**: @critical , @p0 , @p1
 - Nach **Datenbedarf**: @needs-db , @needs-api

Tests mit Tags in Playwright implementieren

- **Option 1: Tags im Testtitel**

```
[TestMethod]
[Category("auth")]
[Category("smoke")]
public async Task BenutzerKannEinloggen()
{
    // Test-Implementierung
}
```

- **Option 2: Tags über Konfigurationsobjekt**

```
[TestMethod]
[Category("api")]
public async Task ApiAntwortEnthaelteKorrekteDaten()
{
    // Test-Implementierung
}
```

- **Option 3: Tags für Testgruppen**

```
[TestClass]
[Category("payment")]
```


Tagbasierte Teststrategie: Beispiele nach Typ

```
1 // 1. API-Tests taggen
2 [TestMethod]
3 [Category("api")]
4 public async Task ApiGibtKorrekteBenutzerdatenZurueck()
5 {
6     var response = await Request.GetAsync("/api/users/me");
7     Assert.IsTrue(response.Ok);
8     var data = await response.JsonAsync<dynamic>();
9     Assert.IsNotNull(data.id);
10 }
11
12 // 2. Visuelle Tests taggen
13 [TestMethod]
14 [Category("visual")]
15 [Category("ui")]
16 public async Task DashboardZeigtKorrekteKomponentenAn()
17 {
18     await Page.GotoAsync("/dashboard");
19     await Expect(Page).ToHaveScreenshotAsync("dashboard.png");
20 }
```

21

Tagbasierte Teststrategie: Tests nach Tags ausführen

- **Nur bestimmte Tags ausführen:**

```
# Nur Smoke-Tests ausführen
dotnet test --filter Category=smoke

# Nur API-Tests ausführen
dotnet test --filter Category=api
```

- **Bestimmte Tags ausschließen:**

```
# Alle Tests außer langsame Tests ausführen
dotnet test --filter "Category≠slow"
```

- **Kombinierte Tag-Filterung:**

```
# Nur Tests, die sowohl critical als auch ui markiert sind
dotnet test --filter "Category=critical&Category=ui"

# Tests, die entweder smoke oder fast sind
dotnet test --filter "Category=smoke|Category=fast"
```

Tagbasierte Teststrategie in CI/CD-Pipelines

```
1  # Beispiel GitHub Actions Workflow
2  name: Playwright Tests
3  on: push
4  jobs:
5    api-tests:
6      name: API Tests
7      runs-on: ubuntu-latest
8      steps:
9        - uses: actions/checkout@v3
10       - uses: actions/setup-dotnet@v3
11         with:
12           dotnet-version: '6.0'
13       - name: Restore dependencies
14         run: dotnet restore
15       - name: Install Playwright
16         run: pwsh bin/Debug/net6.0/playwright.ps1 install --with-deps
17       - name: Run API tests
18         run: dotnet test --filter Category=api
```

Tagbasierte Teststrategie in der Konfiguration

```
1  <!-- .runsettings or test project configuration -->
2  <RunSettings>
3    <TestRunParameters>
4      <!-- Smoke tests configuration -->
5      <Parameter name="TestCategory" value="smoke" />
6      <Parameter name="BrowserType" value="chromium" />
7    </TestRunParameters>
8  </RunSettings>
9
10 // Alternative: Use multiple test projects or MSBuild conditions
11 // Example in .csproj:
12 // <ItemGroup Condition="'$(TestCategory)' = 'smoke'">
13 //   <TestCategory Include="smoke" />
14 // </ItemGroup>
```

Best Practices für Test-Tagging

- **Konsistente Namenskonvention**
 - Eindeutiges Präfix (@ wird empfohlen)
 - Klare, aussagekräftige Tag-Namen
 - Dokumentierte Bedeutungen im Projekt
- **Organisatorische Struktur**
 - Kombiniere mehrere Kategorien für präzise Filterung
 - Beispiel: @ui @auth @critical für kritische UI-Authentifizierungstests
 - Halte die Tag-Anzahl überschaubar (max. 3-5 pro Test)
- **Dynamische Test-Organisation**
 - Smoke-Tests: @smoke für schnelle Verifikation nach Deployments
 - Feature-basierte Tags: @feature-xyz für Tests einer neuen Funktion
 - Release-Tags: @v2.0 für Version-spezifische Tests
- **CI/CD-Integration**

Diskussion: Testing-Strategien

Welche Strategien passen zu eurem Projekt?

Visuelle Regression mit Playwright

- **Was ist visuelle Regression?**
 - Automatischer Vergleich von Screenshots, um unbeabsichtigte UI-Änderungen zu erkennen
 - Ergänzt klassische Assertions um visuelle Prüfungen
- **Typische Anwendungsfälle:**
 - UI-Refactorings, Design-Updates, Regressionstests

Visuelle Vergleiche mit Playwright

- **Screenshot-Assertion:**

- `await Expect(Page).ToHaveScreenshotAsync()`
- Erstellt beim ersten Lauf einen Referenz-Screenshot ("golden file")
- Vergleicht bei Folgeläufen gegen die Referenz

- **Vergleich von Text/Binärdaten:**

- `await Expect(value).ToMatchSnapshotAsync()`
- Für Text, JSON, API-Responses, etc.

Beispiel: Screenshot- und Visual Testing

```
1  using Microsoft.Playwright.MSTest;
2
3  [TestClass]
4  public class VisualTests : PageTest
5  {
6      [TestMethod]
7      public async Task LandingpageVisuellUnveraendert()
8      {
9          await Page.GotoAsync("https://playwright.dev");
10
11          // Option 1: Playwright's built-in visual comparison
12          await Expect(Page).ToHaveScreenshotAsync("landing-page.png");
13      }
14
15      [TestMethod]
16      public async Task KomponentenScreenshot()
17      {
18          await Page.GotoAsync("http://localhost:3000");
19
20          // Screenshot von spezifischem Element
21          var headerElement = Page.Locator("#h1").First();
```

ARIA Snapshots - Accessibility Struktur testen

TestMethod

```
1  public async Task NavigationAriaStruktur()  
2  {  
3      await Page.GotoAsync("http://localhost:3000");  
4  
5      // ARIA-Snapshot der Navigation erstellen  
6      var navElement = Page.GetByRole(AriaRole.Navigation);  
7      await Expect(navElement).ToMatchAriaSnapshotAsync(@"  
8          - navigation:  
9              - link ""Home""  
10             - link ""News""  
11             - link ""About""  
12      ");  
13  }  
14  
15  [TestMethod]  
16  public async Task FormularAriaStruktur()  
17  {  
18      await Page.GotoAsync("http://localhost:3000/auth/signin");  
19  
20      // Formular-Accessibility testen  
21      await Expect(Page.Locator("form")).ToMatchAriaSnapshotAsync(@"
```

Optionen & Best Practices

- **Toleranzen einstellen:**

- `MaxDiffPixels` , `MaxDiffPixelRatio` für erlaubte Unterschiede
- **Beispiel:** `await Expect(Page).ToHaveScreenshotAsync(new() { MaxDiffPixels = 100 })`

- **Dynamische Inhalte ausblenden:**

- Mit `StylePath` eigene CSS-Regeln beim Screenshot anwenden
- **Beispiel:** `await Expect(Page).ToHaveScreenshotAsync(new() { StylePath = "./screenshot.css" })`

- **Globale Defaults:**

- In `.runsettings` oder Test-Konfiguration unter Screenshot-Einstellungen

Snapshot-Verwaltung & Stabilität

- **Snapshots werden im Ordner** `<testfile>-snapshots/` **gespeichert**
- **Snapshots sollten versioniert werden (z.B. mit git)**
- **Aktualisieren bei UI-Änderungen:**
 - Mit `dotnet test -- --update-snapshots` (oder equivalent .NET command)
- **Stabilität:**
 - Immer im gleichen OS, Browser, Headless/Headful-Modus testen
 - Fonts, Rendering, Hardware können Unterschiede verursachen

Übung 7 – Visuelle Regression testen

Ziel: Visuelle Regressionstests für eine Seite oder Komponente einführen.

Aufgaben:

1. Schreibe einen Test mit `ToHaveScreenshotAsync()` für eine Seite oder ein UI-Element. 2. Führe den Test aus und prüfe, ob ein Snapshot erstellt wird. 3. Simuliere eine visuelle Änderung und führe den Test erneut aus. 4. Aktualisiere den Snapshot mit `dotnet test -- --update-snapshots` und prüfe das Ergebnis.

Zeit: 25 Minuten

Spezialthemen

Warum Mobile Testing?

- **Mobiler Traffic** dominiert das Web
- **Responsive Design** ist Standard
- **Unterschiedliche Geräte** zeigen unterschiedliche Probleme
- **Gerätespezifische Features** müssen getestet werden
- **Performance-Unterschiede** zwischen Desktop und Mobil

Mobile-First Testing

- **Mobile-First Entwicklung** erfordert Mobile-First Testing
- **Kritische Nutzerpfade** auf Mobilgeräten priorisieren
- **Eingeschränkte Bildschirmgröße** beeinflusst UI-Design
- **Touch-basierte Interaktionen** statt Maus und Tastatur
- **Netzwerkbedingungen** simulieren (3G, 4G, instabile Verbindungen)
- **Offline-Funktionalität** testen

Mobile vs. Desktop Unterschiede

Mobile Herausforderungen

- Kleinere Bildschirme
- Touch-Interaktionen
- Virtuelle Tastatur
- Gerätesensoren (GPS, Beschleunigung)
- Batterieverbrauch
- Unterschiedliche Browser-Engines
- Langsamere Prozessoren

Desktop Vorteile

- Größere Bildschirme
- Präzise Maussteuerung
- Physische Tastatur
- Konsistentere Performance
- Stabilere Netzwerkverbindung
- Erweiterte Debugging-Tools
- Mehr Rechenleistung

Live-Demo: Mobile Testing

Praktische Demonstration der Konzepte

Geräte-Emulation in Playwright - Konfiguration

```
1  <!-- In .runsettings or similar .NET configuration -->
2  <configuration>
3    <TestRunParameters>
4      <!-- Desktop Chrome -->
5      <Parameter name="BrowserType" value="chromium" />
6      <Parameter name="DeviceType" value="Desktop Chrome" />
7
8      <!-- iPhone 13 -->
9      <!-- <Parameter name="BrowserType" value="webkit" /> -->
10     <!-- <Parameter name="DeviceType" value="iPhone 13" /> -->
11
12     <!-- Galaxy S8 -->
13     <!-- <Parameter name="BrowserType" value="chromium" /> -->
14     <!-- <Parameter name="DeviceType" value="Galaxy S8" /> -->
15   </TestRunParameters>
16 </configuration>
```

Geräte-Emulation im Test

```
1 // Direkt im Test verwenden
2 [TestMethod]
3 public async Task TestAufiPhone()
4 {
5     var context = await Browser.NewContextAsync(Devices.IPhone13);
6     var page = await context.NewPageAsync();
7     // Test läuft jetzt als wäre er auf einem iPhone
8
9     // Cleanup
10    await page.CloseAsync();
11    await context.CloseAsync();
12 }
```

Benutzerdefinierte Geräte-Emulation

```
1  // Benutzerdefiniertes Gerät erstellen
2  var myCustomPhone = new BrowserNewContextOptions
3  {
4      UserAgent = "Mozilla/5.0 (Linux; Android 11; Custom Phone) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/96.0.4664.110 Mobile Saf
5      ViewportSize = new ViewportSize
6      {
7          Width = 412,
8          Height = 915
9      },
10     DeviceScaleFactor = 2.625f,
11     IsMobile = true,
12     HasTouch = true
13 };
```

Benutzerdefinierte Geräte verwenden

```
1 // Im Test verwenden
2 [TestMethod]
3 public async Task TestAufBenutzerdefiniertemGeraet()
4 {
5     var contextOptions = new BrowserNewContextOptions(myCustomPhone)
6     {
7         Geolocation = new Geolocation { Longitude = 48.8584f, Latitude = 2.2945f },
8         Permissions = new[] { "geolocation" },
9         Locale = "de-DE",
10        TimezoneId = "Europe/Berlin"
11    };
12
13    var context = await Browser.NewContextAsync(contextOptions);
14    var page = await context.NewPageAsync();
15    // Test mit benutzerdefinierten Einstellungen
16
17    // Cleanup
18    await page.CloseAsync();
19    await context.CloseAsync();
20 }
```

Emulierte Funktionen - Teil 1

```
1  // Erweiterte Geräteeigenschaften emulieren
2  [TestMethod]
3  public async Task TestMitEmuliertenFunktionen()
4  {
5      var contextOptions = new BrowserNewContextOptions(Devices.IPhone13)
6      {
7          // Geolocation
8          Geolocation = new Geolocation { Latitude = 52.520008f, Longitude = 13.404954f },
9          Permissions = new[] { "geolocation" },
10
11          // Sprache und Zeitzone
12          Locale = "de-DE",
13          TimezoneId = "Europe/Berlin"
14      };
15
16      var context = await Browser.NewContextAsync(contextOptions);
17      var page = await context.NewPageAsync();
18      await page.GotoAsync("https://example.com");
19
20      // Cleanup
21      await page.CloseAsync();
```

Emulierte Funktionen - Teil 2

```
1 // Weitere emulierte Funktionen
2 [TestMethod]
3 public async Task TestMitWeiterenEmuliertenFunktionen()
4 {
5     var contextOptions = new BrowserNewContextOptions(Devices.IPhone13)
6     {
7         // Farbschema
8         ColorScheme = ColorScheme.Dark, // oder ColorScheme.Light
9         // Offline-Modus
10        Offline = false,
11
12        // HTTP-Credentials
13        HttpCredentials = new HttpCredentials
14        {
15            Username = "user",
16            Password = "pass"
17        },
18
19        // Kamera/Mikrofon-Berechtigungen
20        Permissions = new[] { "camera", "microphone" }
21    };
```


Responsive Tests schreiben - Desktop & Mobile

```
1 // Responsive Tests mit verschiedenen Viewports
2 [TestClass]
3 public class ResponsiveTests : PageTest
4 {
5     [TestMethod]
6     public async Task NavigationAufDesktop()
7     {
8         await Page.SetViewportSizeAsync(1280, 800);
9         await Page.GotoAsync("/");
10
11         // Desktop-Navigation prüfen
12         await Expect(Page.GetByTestId("desktop-menu")).ToBeVisibleAsync();
13         await Expect(Page.GetByTestId("mobile-menu")).ToBeHiddenAsync();
14     }
15
16     [TestMethod]
17     public async Task NavigationAufMobilgeraeten()
18     {
19         await Page.SetViewportSizeAsync(375, 667);
20         await Page.GotoAsync("/");
21     }
22 }
```

Breakpoint-basierte Tests - Definition

```
1  // Breakpoints definieren
2  var breakpoints = new Dictionary<string, ViewportSize>
3  {
4      ["xs"] = new() { Width = 320, Height = 568 }, // iPhone SE
5      ["sm"] = new() { Width = 375, Height = 667 }, // iPhone 8
6      ["md"] = new() { Width = 768, Height = 1024 }, // iPad
7      ["lg"] = new() { Width = 1024, Height = 768 }, // iPad Landscape
8      ["xl"] = new() { Width = 1280, Height = 800 }, // Desktop
9      ["xxl"] = new() { Width = 1920, Height = 1080 } // Large Desktop
10 };
```

Breakpoint-basierte Tests - Implementierung

```
1 // Tests für alle Breakpoints
2 [DynamicData(nameof(GetBreakpoints), DynamicDataSourceType.Method)]
3 public async Task NavigationAufBreakpoint(string name, ViewportSize viewport)
4 {
5     await Page.SetViewportSizeAsync(viewport.Width, viewport.Height);
6     await Page.GotoAsync("/");
7
8     // Responsive Verhalten prüfen
9     if (viewport.Width < 768)
10    {
11        // Mobile Verhalten
12        await Expect(Page.GetByTestId("mobile-menu")).ToBeVisibleAsync();
13        await Expect(Page.GetByTestId("desktop-menu")).ToBeHiddenAsync();
14    }
15    else
16    {
17        // Desktop Verhalten
18        await Expect(Page.GetByTestId("desktop-menu")).ToBeVisibleAsync();
19        await Expect(Page.GetByTestId("mobile-menu")).ToBeHiddenAsync();
20    }
21 }
```

Touch-Events simulieren - Tap & Swipe

```
1  // Touch-Events testen
2  [TestMethod]
3  public async Task TouchInteraktionen()
4  {
5      // iPhone 13 emulieren
6      await Page.SetViewportSizeAsync(390, 844);
7      await Page.GotoAsync("/");
8
9      // Tap (einfacher Touch)
10     await Page.GetByRole(AriaRole.Button, new() { Name = "Menü" }).TapAsync();
11
12     // Swipe (Touch mit Bewegung)
13     var carousel = Page.Locator(".carousel");
14     var boundingBox = await carousel.BoundingBoxAsync();
15
16     if (boundingBox != null)
17     {
18         // Swipe von rechts nach links
19         await Page.Touchscreen.TapAsync(
20             boundingBox.X + boundingBox.Width - 10,
21             boundingBox.Y + boundingBox.Height / 2
```

Übung 6A – Responsive Design Testing (Feed App)

Ziel: Umfassende Tests für responsive Design mit Emulation & Visual Testing.

Aufgaben:

1. Responsive Layouts für verschiedene Viewports testen (Navigation, Grid).
2. Touch-Interaktionen (`tap` , Swipe/Pull-to-Refresh) simulieren.
3. Visuelle Tests mit `ToHaveScreenshotAsync` für Seiten und Komponenten durchführen.

Zeit: 45 Minuten

Nachbesprechung: Mobile & Responsive

Diskussion der Übungsergebnisse

Parameterized Testing – Einführung

- **Wiederholende Tests mit unterschiedlichen Daten** durchführen
- **DRY-Prinzip:** Don't Repeat Yourself – Testcode reduzieren
- **Datengetrieben:** Tests mit verschiedenen Eingaben und erwarteten Ergebnissen
- **Flexibel:** Nutzbar sowohl auf Test- als auch Projektebene
- **Leistungsfähig:** Kombinierbar mit Fixtures und Hooks

Parameterized Testing – Anwendungsfälle

- **Formularvalidierung** mit verschiedenen Eingabewerten
- **API-Endpunkte** mit unterschiedlichen Parametern
- **Responsive Design** für verschiedene Gerätetypen
- **Mehrsprachige UIs** mit verschiedenen Locales
- **Berechtigungssysteme** mit verschiedenen Benutzerrollen

Grundlegende Test-Parameterisierung

```
1 // Daten-Array definieren und testen
2 public static IEnumerable<object[]> GetGreetingTestCases()
3 {
4     yield return new object[] { "Alice", "Hallo, Alice!" };
5     yield return new object[] { "Bob", "Hallo, Bob!" };
6     yield return new object[] { "Charlie", "Hallo, Charlie!" };
7 }
8
9 [DynamicData(nameof(GetGreetingTestCases), DynamicDataSourceType.Method)]
10 public async Task BegruessungFuer(string name, string expected)
11 {
12     await Page.GotoAsync($"https://example.com/greet?name={name}");
13     await Expect(Page.GetByRole(AriaRole.Heading)).ToHaveTextAsync(expected);
14 }
```

- **Array von Testfällen** definiert
- **forEach-Schleife** generiert Tests dynamisch
- Einzigartiger **Testname** durch Template-String
- **Gleiche Testlogik** für alle Testfälle

Test-Hooks mit Parameterisierung

```
1 // Hooks außerhalb der foreach-Schleife: werden nur einmal ausgeführt
2 public class UserRoleTests : PageTest
3 {
4     [TestInitialize]
5     public async Task Setup()
6     {
7         await Page.GotoAsync("https://example.com");
8     }
9
10    // Testfälle
11    public static IEnumerable<object[]> GetUserRoleTestCases()
12    {
13        yield return new object[] { "Admin", new[] { "create", "read", "update", "delete" } };
14        yield return new object[] { "Editor", new[] { "create", "read", "update" } };
15        yield return new object[] { "Viewer", new[] { "read" } };
16    }
17
18    [DynamicData(nameof(GetUserRoleTestCases), DynamicDataSourceType.Method)]
19    public async Task BenutzerrolleHatKorrekteBerechtigungen(string name, string[] permissions)
20    {
21        // Login als Benutzer mit dieser Rolle
```

Test-Hooks pro Testfall

```
1 // Hooks innerhalb eines describe-Blocks: werden für jeden Testfall ausgeführt
2 public class UserRolePageAccessTests : PageTest
3 {
4     private string _role = string.Empty;
5     private string[] _expectedPages = Array.Empty<string>();
6
7     public static IEnumerable<object[]> GetUserRolePageTestCases()
8     {
9         yield return new object[] { "admin", new[] { "Dashboard", "Benutzer", "Einstellungen" } };
10        yield return new object[] { "editor", new[] { "Dashboard", "Inhalte" } };
11        yield return new object[] { "viewer", new[] { "Dashboard" } };
12    }
13
14    private async Task LoginWithRoleAsync(string role)
15    {
16        await Page.GotoAsync("https://example.com/login");
17        await Page.GetByLabel("Benutzername").FillAsync(role);
18        await Page.GetByLabel("Passwort").FillAsync("password123");
19        await Page.GetByRole(AriaRole.Button, new() { Name = "Login" }).ClickAsync();
20    }
21 }
```

Projektebene Parameterisierung – Definition

```
1  // UserRoleTestBase.cs - Benutzerdefinierte Test-Basisklasse
2  using Microsoft.Playwright.MSTest;
3  using Microsoft.VisualStudio.TestTools.UnitTesting;
4
5  public abstract class UserRoleTestBase : PageTest
6  {
7      protected string Role { get; private set; } = "user"; // Default-Wert
8
9      [TestInitialize]
10     public virtual void SetupRole()
11     {
12         // Parameter aus Umgebungsvariable oder Konfiguration lesen
13         Role = Environment.GetEnvironmentVariable("USER_ROLE") ?? "user";
14     }
15 }
```

- **Abstract Base Class:** Stellt gemeinsame Funktionalität bereit
- **Protected Properties:** Können von abgeleiteten Klassen verwendet werden
- **Environment Variables:** Ermöglichen Konfiguration zur Laufzeit

Projektebene Parameterisierung – Verwendung

```
1  // .runsettings - .NET-Testkonfiguration
2  <RunSettings>
3    <TestRunParameters>
4      <!-- Admin Tests -->
5      <!-- Parameter name="USER_ROLE" value="admin" / -->
6
7      <!-- Editor Tests -->
8      <!-- Parameter name="USER_ROLE" value="editor" / -->
9
10     <!-- Viewer Tests -->
11     <!-- Parameter name="USER_ROLE" value="viewer" / -->
12   </TestRunParameters>
13 </RunSettings>
14
15 // Alternative: Separate test projects or configuration profiles
16 // dotnet test --environment Development -- TestRunParameters.Parameter\ (name="USER_ROLE", value="admin")\
```

- Mehrere **Projekt-Konfigurationen** definieren
- Jedes Projekt erhält einen **anderen Parameterwert**
- Tests laufen mit **jeder Konfiguration** durch
- Parameterwerte über **use-Objekt** gesetzt

Parameter in Fixtures verwenden

```
1  // UserRoleFixtures.cs
2  using Microsoft.Playwright.MSTest;
3  using Microsoft.VisualStudio.TestTools.UnitTesting;
4
5  public class UserRoleFixtures : UserRoleTestBase
6  {
7      protected UserInfo User { get; private set; } = new();
8
9      [TestInitialize]
10     public override void SetupRole()
11     {
12         base.SetupRole();
13
14         // Benutzerdetails basierend auf der Rolle erstellen
15         var userMap = new Dictionary<string, UserInfo>
16         {
17             ["admin"] = new() { Name = "Admin User", Email = "admin@example.com", IsAdmin = true },
18             ["editor"] = new() { Name = "Editor User", Email = "editor@example.com", IsAdmin = false },
19             ["user"] = new() { Name = "Regular User", Email = "user@example.com", IsAdmin = false }
20         };
21 }
```

Einsatz in Tests

```
1 // UserPermissionTests.cs
2 public class UserPermissionTests : UserRoleFixtures
3 {
4     [TestMethod]
5     public async Task BenutzerberechtigungenBasierendAufRolle()
6     {
7         await Page.GotoAsync("/dashboard");
8
9         // Parameter und abgeleitete Fixture in Tests verwenden
10        Console.WriteLine($"Teste als {Role} mit Namen {User.Name}");
11
12        // Administratorbereich nur für Admins sichtbar
13        if (User.IsAdmin)
14        {
15            await Expect(Page.GetByRole(AriaRole.Link, new() { Name = "Admin-Panel" })).ToBeVisibleAsync();
16        }
17        else
18        {
19            await Expect(Page.GetByRole(AriaRole.Link, new() { Name = "Admin-Panel" })).ToBeHiddenAsync();
20        }
21    }
22 }
```

- Test nutzt sowohl **direkten Parameter** (role) als auch **abgeleitete Fixture** (user)

Daten aus CSV-Dateien einlesen

```
1  // CsvFormValidationTests.cs
2  using Microsoft.Playwright.MSTest;
3  using Microsoft.VisualStudio.TestTools.UnitTesting;
4  using CsvHelper;
5  using System.Globalization;
6
7  public class CsvFormValidationTests : PageTest
8  {
9      public static IEnumerable<object[]> GetCsvTestCases()
10     {
11         using var reader = new StringReader(File.ReadAllText("testdaten.csv"));
12         using var csv = new CsvReader(reader, CultureInfo.InvariantCulture);
13
14         var records = csv.GetRecords<FormTestData>().ToList();
15
16         foreach (var record in records)
17         {
18             yield return new object[] { record };
19         }
20     }
21 }
```

Übung 6 – Parameterized Testing

Ziel: Tests mit verschiedenen Testdaten und Konfigurationen ausführen.

Aufgaben:

1. Test-Ebene Parameterisierung:

- Erstelle einen parameterisierten Test mit mindestens 3 Testfällen.
- Nutze verschiedene Suchbegriffe/Filter in der News-App.

2. Projekt-Ebene Parameterisierung:

- Definiere ein benutzerdefiniertes Test-Fixture für verschiedene Benutzerrollen.
- Konfiguriere mehrere Projekte in der Playwright-Konfiguration.
- Implementiere einen Test, der sich basierend auf der Rolle unterschiedlich verhält.

Zeit: 35 Minuten

Nachbesprechung: Parameterized Testing

Diskussion der Übungsergebnisse

Shadow DOM und Web Components

```
1 // Shadow DOM Testing
2 [TestMethod]
3 public async Task ShadowDomElemente()
4 {
5     await Page.GotoAsync("/page-with-web-components");
6
7     // Auf Shadow DOM zugreifen
8     var shadowHost = Page.Locator("my-custom-element");
9
10    // In Shadow DOM navigieren
11    var shadowButton = shadowHost.Locator("pierce/button.shadow-button");
12    await shadowButton.ClickAsync();
13
14    // Alternativ mit CSS-Locator
15    await Page.Locator("my-custom-element >>> button.shadow-button").ClickAsync();
```

- **pierce/**: Spezieller Präfix für Shadow DOM-Durchdringung
- **>>>**: Alternative Syntax für Shadow DOM-Locators

Performance Testing mit Playwright

- **Warum Performance wichtig ist:** Benutzererfahrung, Conversion, SEO
- **Was kann gemessen werden:** Ladezeiten, Rendering, Interaktivität
- **Wie Playwright hilft:** Zugriff auf Browser-Performance-APIs, DevTools Protocol
- **Anwendungsfälle:** Regressionstests, Überwachung, Optimierung

Performance-Metriken: Web Vitals

- **Core Web Vitals:** Wichtigste Nutzer-zentrierte Metriken
 - **LCP (Largest Contentful Paint):** Ladeperformance
 - **FID (First Input Delay):** Interaktivität (in Tests: TBT - Total Blocking Time)
 - **CLS (Cumulative Layout Shift):** Visuelle Stabilität
- **Weitere Metriken:**
 - **TTFB (Time to First Byte):** Server-Antwortzeit
 - **FCP (First Contentful Paint):** Wann erster Inhalt erscheint
 - **TTI (Time to Interactive):** Wann die Seite vollständig interaktiv ist

Performance-APIs in Playwright

```
1  using Microsoft.Playwright.MSTest;
2  using Microsoft.VisualStudio.TestTools.UnitTesting;
3
4  [TestMethod]
5  public async Task NavigationPerformanceMessen()
6  {
7      await Page.GotoAsync("https://google.com");
8
9      var navigationTiming = await Page.EvaluateAsync<dynamic[]>(
10         "() => performance.getEntriesByType('navigation')")
11     );
12
13     Assert.Less(navigationTiming[0].duration, 1000);
14 }
```

TestMethod

```
1  public async Task LcpUndClsMessen()  
2  {  
3      await Page.GotoAsync("https://google.com");  
4  
5      var paintMetrics = await Page.EvaluateAsync<dynamic[]>(  
6          "() => window.performance.getEntriesByType('paint')"  
7      );  
8  
9      var firstPaint = paintMetrics[0];  
10     var firstContentfulPaint = paintMetrics[1];  
11  
12     Assert.Less(firstPaint.startTime, 1000);  
13     Assert.Less(firstContentfulPaint.startTime, 600);  
14 }
```


Performance-Metriken mit Browser APIs

TestMethod

```
1  public async Task PerformanceMetrikenSammeln()
2  {
3      // Hinweis: Network.emulateNetworkConditions hat bekannte Zuverlässigkeitsprobleme
4      // Für Netzwerk-Emulation besser Playwright's route() verwenden oder System-Level Tools
5
6      await Page.GotoAsync("https://example.com/");
7
8      // Performance-Metriken sammeln mit Browser Performance API
9      var metrics = await Page.EvaluateAsync<dynamic>(@"
10      () => {
11          const entries = performance.getEntriesByType('navigation')[0];
12          return {
13              loadTime: entries.loadEventEnd - entries.loadEventStart,
14              domContentLoaded: entries.domContentLoadedEventEnd - entries.navigationStart,
15              firstByte: entries.responseStart - entries.navigationStart
16          };
17      }
18      ");
19
20      // Performance-Thresholds prüfen
21  }
```

Performance-Monitoring: Best Practices

Tipps für Tests

- Testen in CI mit konsistenten Umgebungen
- Testen mit unterschiedlichen Netzwerkprofilen
- Performance-Budgets definieren
- Baseline erstellen und Regressionen tracken
- Headless-Modus für konsistente Ergebnisse

Messwerte nutzen

- Trends über Zeit wichtiger als Einzelwerte
- Core Web Vitals priorisieren
- Kontext beachten (Desktop vs. Mobile)
- Visuelle Metriken (LCP, CLS) mit technischen (TTFB, JS-Zeit) kombinieren
- Nutzerbasierte Feldmessungen ergänzen

Ausblick: Continuous Integration (CI/CD)

- **Ziel:** Tests automatisch bei Code-Änderungen ausführen.
- **Vorteile:** Frühes Feedback, Qualitätssicherung, weniger manuelle Tests.
- **Tools:** GitHub Actions, GitLab CI, Jenkins, CircleCI etc.
- **Playwright in CI:** Headless-Modus, parallele Ausführung, Reporting.

Beispiel: GitHub Actions Workflow

```
1  name: Playwright Tests
2  on: [push]
3  jobs:
4    test:
5      runs-on: ubuntu-latest
6      steps:
7        - uses: actions/checkout@v3
8        - uses: actions/setup-dotnet@v3
9          with:
10            dotnet-version: '6.0'
11        - name: Restore dependencies
12          run: dotnet restore
13        - name: Install Playwright
14          run: pwsh bin/Debug/net6.0/playwright.ps1 install --with-deps
15        - name: Run Playwright tests
16          run: dotnet test
17        - uses: actions/upload-artifact@v3
18          if: always()
19          with:
20            name: playwright-report
21            path: playwright-report/
```

Best Practices & Zusammenfassung

- **Robuste Locators:** `getByRole` , `getByText` , `getByTestId` bevorzugen.
- **Auto-Wait nutzen:** Playwright wartet automatisch, `waitFor` selten nötig.
- **Struktur:** POM, Fixtures für Wartbarkeit und Wiederverwendbarkeit.
- **Isolation:** Storage State, API Mocking, Testdaten-Management.
- **Debugging:** Inspector, Trace Viewer sind mächtige Werkzeuge.
- **CI/CD:** Tests früh und oft ausführen.
- **Mobile:** Spezifische Techniken anwenden.

Abschluss & Fragen

- Zusammenfassung der drei Tage
- Wichtigste Erkenntnisse
- Ausblick: Weitere Playwright-Features (Visual Testing, API Testing, Component Testing)
- Offene Fragen & Diskussion

Workshop Abschluss

Zusammenfassung und nächste Schritte



Daniel Sogl - linktr.ee/daniel_sogl

Was haben wir gelernt?

Grundlagen & API

- **E2E-Testing und Test-Automatisierung**
- **Playwright API:** Locators, Aktionen, Assertions
- **Strukturierung:** Page Objects, Fixtures
- **Wiederverwendbarkeit:** Komponenten, Patterns

Fortgeschrittene Features

- **Authentifizierung und Session-Handling**
- **API-Mocking und Netzwerk-Interception**
- **Visuelle Tests und Screenshots**
- **Mobile Testing und Geräte-Emulation**
- **CI/CD-Integration:** Tests in der Pipeline

Nächste Schritte

Technische Schritte

- **Testabdeckung erweitern:** Mehr kritische Pfade
- **Refactoring:** Tests verbessern und wartbarer machen
- **CI/CD-Integration:** Tests in den Build-Prozess

Organisatorische Schritte

- **Team-Schulung:** Weitergabe des Wissens
- **Testautomatisierung-Strategie:** Langfristiger Plan
- **Monitoring:** Testergebnisse verfolgen und analysieren

Ressourcen für die Weiterbildung

Offizielle Ressourcen

- **Dokumentation:** playwright.dev
- **GitHub:** github.com/microsoft/playwright
- **VS Code Extension:** Playwright für VS Code
- **Workshop-Material:** GitHub-Repository

Community & Updates

- **Discord:** playwright.dev/community/discord
- **GitHub Discussions:** Fragen und Antworten
- **Blog-Artikel:** Regelmäßige Updates
- **YouTube-Tutorials:** Microsoft Developer Channel

Häufige Fehlerquellen

Technische Fehler

- **Locators zu spezifisch:** Anfällig für UI-Änderungen
- **Feste Wartezeiten:**
`Page.WaitForTimeoutAsync()` statt Events
- **Fehlende Assertions:** Keine klare Erfolgsvalidierung

Strukturelle Fehler

- **Übergroße Tests:** Zu viele Schritte in einem Test
- **Unzureichende Isolation:** Tests beeinflussen einander
- **Langsame Tests:** Unnötige Aktionen und Navigationen

Tipps für erfolgreiche Playwright-Tests

Codequalität

- **Kleinhaltung:** Tests auf einen Aspekt fokussieren
- **Wiederverwendbarkeit:** Helper-Funktionen nutzen
- **Gute Locators:** Testdaten-Attribute verwenden
- **Leserliche Tests:** Beschreibende Namen

Testqualität

- **Saubere Umgebung:** Status zwischen Tests zurücksetzen
- **Frühe Fehlererkennung:** Assertions früh platzieren
- **Debugging-Informationen:** Screenshots bei Fehlern
- **Deterministische Tests:** Keine zufälligen Daten

Kontakt & Support

Kontaktmöglichkeiten

- E-Mail: me@danielsogl.de
- Twitter: [@sogldaniel](https://twitter.com/sogldaniel)
- LinkedIn: linkedin.com/in/sogldaniel
- GitHub: github.com/danielsogl

Weitere Informationen

- Website: danielsogl.de
- Blog: Artikel zu Playwright & Testing
- Workshops: Weitere Trainingsangebote
- Consulting: Individuelle Beratung

Vielen Dank!

Fragen? Kommentare? Anregungen?

